Building Intelligent Applications with Spring AI

By John Blum
2024 September 22

Part 1: AI From The Beginning

What was once science fiction has now become reality. From smartphones to smart homes and digital assistants, such as Siri, Google Assistant and Alexa, to autonomous, auto-sensing vehicles and robots that can dance, or learn to play a sport [See: 60 Minutes: Artificial Intelligence], Artificial Intelligence is everywhere.

Humankind has always had a certain fascination and curiosity with superhuman capabilities and hyperintelligence, from superheroes to supercomputers. We have an insatiable desire to unlock the secrets of the mind and tap into the unrealized potential of human abilities. Indeed, one of my favorite superheroes of all time is Tony Stark as Ironman. His brilliant mind and ingenuity are truly inspirational.

But, what makes someone, or something, intelligent? Is it genius, talent or simply hard work?

Thomas Edison once said, "*Genius is 1% inspiration and 99% perspiration.*" Those who dared to question and challenge the status quo brought about marvelous innovations today that humankind could only dream about yesterday.

For centuries, humans have tried to build better and better tools to harvest the resources of our planet in the most economical manner possible. AI is yet another tool, unlike any other, set to accelerate the pace of innovation and our ability to harvest information. From this, we will derive intelligence creating meaningful new insights and actions with unimaginable results.

We begin our exploration of Artificial Intelligence (AI) by looking through the lens of Spring AI and how we might use it to solve real world problems. Along the way we'll discover the capabilities and limitations of AI, as well as its possibilities.

With this book, I have created examples using Spring AI to help facilitate your learning process and success. I hope you enjoy reading this book as much as I did writing it.

Chapter 1 - Getting Started With Spring AI

From the very beginning, Spring has focused on developer productivity by removing complexity through following object-oriented fundamentals in a well structured and intuitive manner.

As software engineers, we want to create amazing software with powerful features that delight our users in the most effective way possible. Artificial Intelligence promises to improve the quality of life for everyone. It can help breathe life into our software. In fact, the best kind of technology is one that is only felt, but never seen.

This reminds me of the Turing Test, "*A test of a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human.*" Indeed, this is at the intersection of where truly powerful software applications will seamlessly blend into everyday life and the full potential of AI can be realized.

Spring has always been at the forefront of pushing the envelope on innovation, oftentimes setting the standard by which everything else is measured. With AI, it is no different, and the Spring team is on a mission. Therefore, when you combine the maturity of Spring with the power of AI, you arrive at something pretty special.

Welcome to Spring AI.

In this chapter, you will:

1. Build your first Spring AI application
2. Learn how to switch AI providers and models
3. And, run the application locally

1.1 - From `start.spring.io` to `spring-boot:run` in 2 minutes

TIP: *All the examples for this book will be in the associated GitHub repository.*

Enough talk. Let's code!

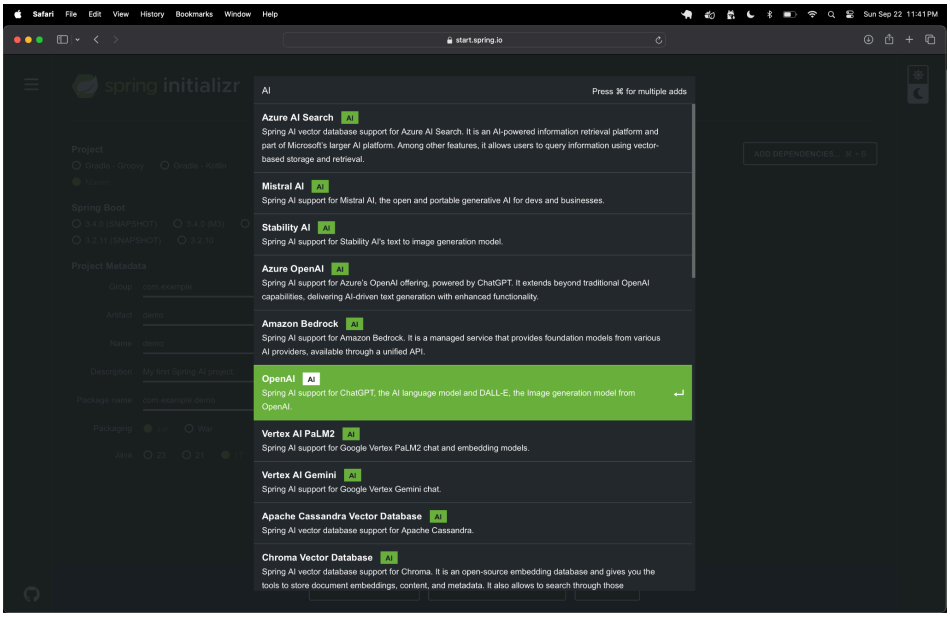
To get started quickly, we will build a really simple Spring AI application that prompts an AI then outputs the generated response.

Like the start of any good Spring project, you begin a Spring AI project by going to Spring Initializer at start.spring.io. You will find a link to Spring Initializer by going into your bookmarks and searching through your top 5. 😉

After you land at Spring Initializer, select your language [*Java, Kotlin, Groovy*] and project [*Maven, Gradle*] of choice. You should always be using the latest GA version of Spring Boot,

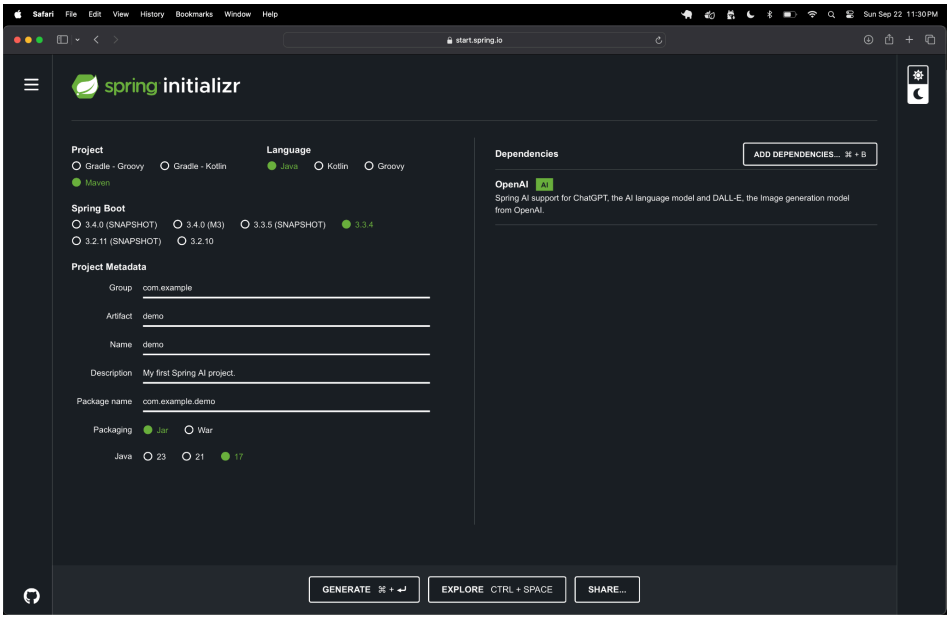
which is 3.3.4 at the time of this writing. Minimally, you should also be using Java 17 when developing in Java. But, the choice is yours, so you choose.

The application will be bundled in a JAR with a dependency on OpenAI. Click on “Add Dependencies” and type in “AI”, or “OpenAI”. In the list that appears, select “OpenAI”:



If you simply typed “AI, you will see that there are many supported AI providers to choose from. We will come back to this topic in a bit.

The end result should look similar to this:



Now, "*generate*" the project.

TIP: *You can preview the contents of the Maven pom.xml or build.gradle file before generating the project by clicking on the "Explore" button.*

Key elements in the Maven POM include:

MAVEN

```
<properties>
  <java.version>17</java.version>
  <spring-ai.version>1.0.0-M2</spring-ai.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
  </dependency>
  <!-- additional dependencies omitted -->
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>${spring-ai.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<repositories>
  <repository>
    <id>spring-milestones</id>
    <name>Spring Milestones</name>
    <url>https://repo.spring.io/milestone</url>
    <snapshots>
      <enabled>false</enabled>
    </snapshots>
  </repository>
</repositories>
```

You can see here that we are using **Java 17** along with **Spring AI** `1.0.0-M2`. Additionally, we included a dependency on Spring AI's OpenAI Spring Boot Starter: `org.springframework.ai:spring-ai-openai-spring-boot-starter`.

Additionally, the generated Maven POM uses Spring AI's BOM (`org.springframework.ai:spring-ai-bom`) for dependency management, controlling the versions of other Spring AI dependencies we may include in our Spring AI project. As a result, it is not necessary to declare the version on the OpenAI starter.

Since we are using a Spring AI milestone version, that is, `1.0.0-M2`, we must declare a reference to Spring's Milestone repository at `repo.spring.io/milestone`.

NOTE: *The Spring team only publishes GA bits to Maven Central.*

Gradle user's can achieve a similar outcome, whether using Groovy or Kotlin syntax, with the following Gradle build file, shown here, in Groovy:

GRADLE [GROOVY]

```
plugins {
  id 'java'
  id 'org.springframework.boot' version '3.3.4'
  id 'io.spring.dependency-management' version '1.1.6'
}

java {
  toolchain {
    languageVersion = JavaLanguageVersion.of(17)
 }
}

repositories {
  mavenCentral()
  maven { url 'https://repo.spring.io/milestone' }
}

ext {
  set('springAiVersion', "1.0.0-M2")
}

dependencies {
 implementation 'org.springframework.ai:spring-ai-openai-spring-boot-starter'
 // …
}

dependencyManagement {
  imports {
    mavenBom "org.springframework.ai:spring-ai-bom:${springAiVersion}"
 }
}
```

TIP: *To find the latest version of Spring AI, open a web browser to spring.io. Under "Projects", navigate to the Spring AI project page. Click on the "LEARN" tab. All of the current and supported versions are listed along with references to documentation and Javadoc.*

TIP: *You can start a new Spring AI project by building a Maven POM from scratch, or by simply modifying an existing Maven POM and including the Spring AI dependencies. If you are reading a digital copy of this book, simply click on this link to generate a new Spring AI project.*

Now, let's look at the generated code. The project contains a basic Spring Boot application:

```
@SpringBootApplication
public class ExampleOneApplication {

   public static void main(String[] args) {
      SpringApplication.run(ExampleOneApplication.class);
   }
}
```

We will change the `main` method to use the `SpringApplicationBuilder` class in order to set `WebApplicationType` to `NONE` before the Spring AI application is bootstrapped.

By default, Spring AI pulls in the `spring-web` dependency, thereby triggering Spring Boot's auto-configuration to start a web container. However, since we are only building a simple CLI application to begin with, we do not need to start a (Tomcat-based) Servlet or (Reactor Netty-based) Reactive web container.

NOTE: *While Spring AI (`org.springframework.ai:spring-ai-core`) makes limited use of `spring-web`, most of the AI provider models supported by Spring AI have a web interface, exposing a REST API. In turn, Spring AI uses `spring-web` to interact with an AI provider's model, such as OpenAI's ChatGPT, using the REST API.*

Our modified `main` method appears as follows:

```
public static void main(String[] args) {

   new SpringApplicationBuilder(ExampleOneApplication.class)
      .web(WebApplicationType.NONE)
      .build()
      .run(args);
}
```

Use of Spring Boot's `SpringApplicationBuilder` class is recommended since it affords you more control over the configuration of the Spring Boot application on startup. You can specify Spring profiles, add additional System properties, or explicitly include other Spring

`@Configuration`, or `@SpringBootConfiguration`, classes that can be bootstrapped on startup, among several other useful functions.

The heart of this Spring AI application is encapsulated in a Spring Boot `ApplicationRunner`, declared as a Spring bean, which prompts the AI and outputs the generated answer:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {

   return args -> {

      String prompt = "In a single word, 'what is the answer to life, the
universe, and everything?'";

      System.out.printf("User> %s%n%n", prompt);

      String response = chatModel.call(prompt);

      System.out.printf("AI> %s%n%n", response);
   };
}
```

However, if you try to run the program now, it throws an Exception:

```
Caused by: java.lang.IllegalArgumentException: OpenAI API key must be set. Use
the connection property: spring.ai.openai.api-key or
spring.ai.openai.chat.api-key property.
```

Spring AI nicely instructs us that we need to set the `spring.ai.openai.api-key` property (or alternatively, `spring.ai.openai.chat.api-key`) to the API Key we created after signing up with OpenAI.

TIP: *Follow OpenAI's Developer quickstart guide from their documentation to get setup and running, using OpenAI's provided models.*

It is recommended that you **do not** declare this property in Spring Boot `application.properties` **nor** as a system environment variable. Remember, this is your personal and private API Key. It was created for you to give you access to AI models provided by OpenAI. Do not share your API Key with anyone or check your API Key into source control, such as GitHub, where it is publicly accessible and visible.

If you ultimately decide to declare your OpenAI API Key in Spring Boot `application.properties`, then I suggest that you:

1) Create a user-specific properties file, such as `application-user.properties`
2) Then, enable the "*user*" profile with `-Dspring.profiles.active=user`

3) Finally, add `application-user.properties` to your `.gitignore` file (Git users) or equivalent exclude file used by your source control management system (SCM).

You will thank me later. 😉

TIP: *It is further recommended that you create and use project-based API Keys along with setting limits on [monthly] usage as a further safe-guard.*

After configuring your OpenAI API Key, re-run the application. You should now see the following:

```
User> In a single word, 'what is the answer to life, the universe, and
everything?'

AI> 42
```

Well, I'll be damned! You don't say! I don't know about you, but I never really bought into what Mahatma Gandhi famously said, "*If it's on the Internet it must be true*", but how can you doubt a thing with "Intelligence" in its name? I mean, really! 😂

We just learned something very valuable here, besides the secret to life, the universe, and, well, "everything"! More importantly, we learned how to shape the generated response of an AI.

The key to the question in our Prompt was, "*In a single word…*". As a result, OpenAI's ChatGPT model responded with "**42**". Now that is pretty cool!

We'll learn more about Prompts, and more generally, Prompt Engineering, later in this book.

NOTE: *See the complete code for this example in GitHub.*

1.2 - Switching AI Provider Models in a Single Step

I was deliberately not specific about which OpenAI model we were using.

As you may be aware, OpenAI offers a wide range of models for different purposes: chat, image generation, audio transcription, speech to text, text to speech, and so on. In some cases, the models are multimodal, which means they can handle multiple forms of input and even generate multiple forms of output or content.

You can specifically control which model from OpenAI you are using by setting the `spring.ai.openai.chat.options.model` Spring AI property to any one of the following enumerated values: `[gpt-4o, gpt-4o-mini, o1-preview, o1-mini, gpt-4-turbo, gpt-3.5-turbo, dall-e-3, tts-1, …]`.

TIP: *To see a complete list of Spring AI supported models, refer to the documentation.*

For example, you can set the chat options Spring AI model property to "`dall-e-3`" if you want to generate images:

```
spring.ai.openai.chat.options.model=dall-e-3
```

The default OpenAI model used by Spring AI is `gpt-4o`.

NOTE: *Unlike your OpenAI API Key, the model property is not sensitive.*

1.2.1 - Switching AI Providers

Of course, you may want to use a model from a completely different AI provider altogether. For example, perhaps you want to use Google Gemini, formerly known as Bard. In this case, simply change your dependency from the OpenAI starter to the VertexAI Gemini starter, like so:

```
<dependency>
   <groupId>org.springframework.ai</groupId>
   <artifactId>spring-ai-vertex-ai-gemini-spring-boot-starter</artifactId>
</dependency>
```

NOTE: *Once again, the dependency version is not required.*

Or, perhaps you want to use Amazon's Bedrock AI, then simply declare:

```
<dependency>
   <groupId>org.springframework.ai</groupId>
   <artifactId>spring-ai-bedrock-ai-spring-boot-starter</artifactId>
</dependency>
```

Using certain models from different AI providers will require you to configure different properties in order to use the model.

For example, to use Google Gemini, you will need to set both the `spring.ai.vertex.ai.gemini.projectId` and `spring.ai.vertex.ai.gemini.location` Spring AI properties in Spring Boot *application.properties*, or an equivalent source.

In turn, this requires you to set up a Google Cloud account and create a new project to which you will associate your use of Google Gemini. Follow the instructions given in Spring AI's documentation to get started. Do not worry, though, this is only required on initial setup.

TIP: *See the complete list of available locations where Google's Vertex AI can be used.*

When we run the application again, this time by using Google Gemini, we see similar, but different output:

```
User> In a single word, 'what is the answer to life, the universe, and
everything?'

AI> Forty-two.
```

The amount of configuration required varies from one AI provider to another. In any case, you always have a wide array of choices when using Spring and Spring AI.

1.3 - Think Global, Build & Run Local

If you are offline, you have an option to run AI locally, on your personal computer, with the help of Ollama. You can conveniently download, install and run Ollama on Linux, Mac or Windows.

After you have installed the software, you need to choose an AI model to run. There are a wide variety of AI models to choose from. Some are even multimodal, like LLaVa. You can also find the available AI models on the official Ollama GitHub project page.

TIP: *To see the available software surrounding the Ollama ecosystem, visit the Ollama GitHub organization. There is support for Python and JavaScript users as well.*

Open a command-line prompt or terminal on your computer, then run:

```
$ ollama run llama3.2
```

NOTE: *Alternatively, run* `ollama run mistral`.

TIP: *See Ollama's llama3.2 documentation for more details.*

Now, as we did when switching between different AI provider models, we can switch to running our Spring AI application using a local AI model by declaring a dependency on the Spring AI's Ollama starter:

```
<dependency>
   <groupId>org.springframework.ai</groupId>
   <artifactId>spring-ai-ollama-spring-boot-starter</artifactId>
</dependency>
```

By default, Spring AI expects the "mistral" AI model to be used. However, you can configure your choice by setting the `spring.ai.ollama.chat.options.model` Spring AI property. For example:

```
spring.ai.ollama.chat.options.model=llama3.2
```

Once more, you should see output similar to:

```
User> In a single word, 'what is the answer to life, the universe, and
everything?'

AI> 42.
```

Magic!

SUMMARY

In this chapter we learned how to get started quickly by going to Spring Initializer at start.spring.io (where all magic on the Internet happens) to create a project and begin building smart applications in Java using Spring AI.

We started with a simple Spring Boot application using Spring AI to prompt OpenAI's ChatGPT, using OpenAI's `gpt-4o` model. We got back a generated response and output the content to standard out.

Next, we saw how we could easily switch to using different AI models provided by an AI provider, such as OpenAI, along with how to switch between different AI providers, including Google Gemini and Amazon Bedrock.

Finally, we explored how to run AI locally, on our personal computer, and use Spring AI with Ollama by interfacing with locally provisioned and running AI models.

Chapter 2 - Understanding Spring AI

Understanding Spring AI is all about learning the features and functionality available and where Spring AI fits into the overall AI landscape. Your focus will then shift to when and how to use the features and functionality of Spring AI to build intelligent applications in Java that can fully leverage AI in the most effective and efficient ways possible.

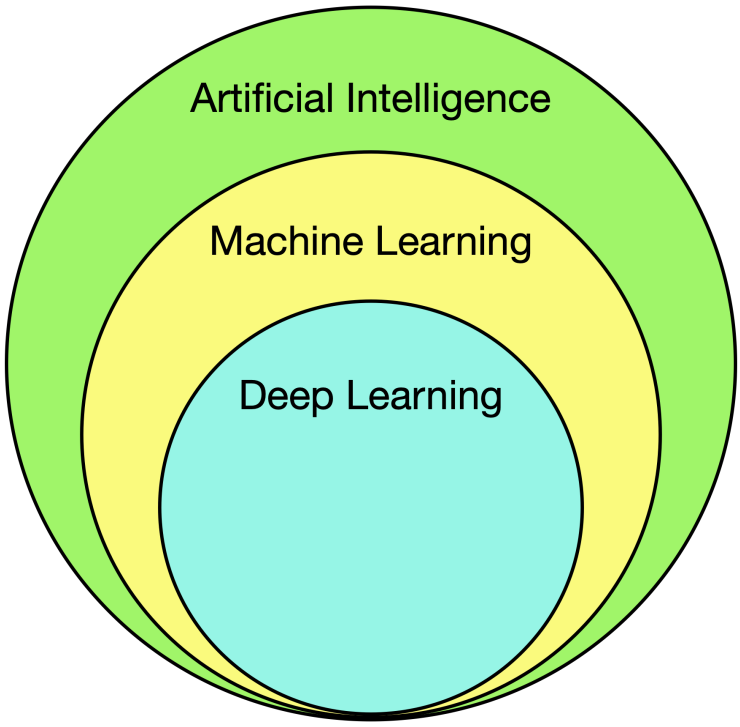
To begin with, it helps to have a fundamental understanding of Artificial Intelligence (AI), and its various disciplines, so that is where we will start. You will see where Spring AI fits into this picture along with what Spring AI is and what it is not.

Let's get started.

2.1 - Brief Overview of AI

The study and development of Artificial Intelligence (AI) has grown since its inception in 1956 to include many different disciplines. Artificial Intelligence is now a term used to generally classify all the various disciplines of AI, such as Machine Learning (ML) and Deep Learning (DL).

Each progressive discipline is a subset of the previous.



TODO

2.2 - Core APIs

Spring AI is rapidly evolving. The features, functionality and capabilities of Spring AI are well-defined in its diverse and extensive application programming interfaces, or APIs.

The cornerstone of Spring AI's APIs, and Spring in general, is its powerful programming model. This programming model is applied consistently across the entire family of Spring projects and all Spring projects strictly adhere to this programming model. Spring AI is no different. As a result, Spring AI's APIs and organizational structure is very intuitive and should feel immediately familiar to you.

TIP: *Spring's programming model is characterized by POJOs and the JavaBeans specification along with many software design patterns. If you are not familiar with Spring's programming model, then learning about JavaBeans and design patterns is a good place to start.*

NOTE: *Developers are more productive when a programming model consistently applies similar constructs across its APIs and libraries. Consistency reinforces familiarity, and is a solid reason to strictly adhere to a programming model in the first place.*

From the very beginning, Spring encouraged programming to interfaces. Interfaces specify the APIs that define the functional capabilities of instances, shielding applications from implementation details, which enables implementations to be interchanged when needed.

What's more, many Spring interfaces extend well beyond simple Java interfaces becoming *Service Provider Interfaces* (SPIs) that enable different provider implementations to be used interchangeably without the adverse effects that can impact your application code. This intrinsic capability truly sets Java frameworks and libraries apart, and Spring is exceptional in this area. Spring exemplifies the *Open/Closed* principle with high-quality.

TIP: *For more information on the fundamental principles of Spring, see Design Philosophy in the core Spring Framework's reference documentation.*

2.2.1 - Chat Client & Model APIs

2.2.1.1 - `ChatModel` API

In the first chapter, you were introduced to the `ChatModel` interface. The example Spring AI application used the `ChatModel` to prompt the AI model with the `call(:String)` method, which returns the response generated by the AI:

```
String response = chatModel.call(prompt);
```

The `prompt` contained a `String` with the chat message, "**`In a single word, 'what is the answer to life, the universe, and everything?'`**", sent to the AI as input.

The `ChatModel` interface defines the most basic and essential operations for "chatting" with any AI provider chat model, such as OpenAI's ChatGPT, or Google's Gemini. In fact, all AI provider implementations supported by Spring AI must conform to and implement this interface.

NOTE: *This effectively makes Spring AI's `ChatModel` interface a Service Provider Interface.*

Most SPIs are *Adapters* for the underlying API published by an AI provider. The provider's APIs may be implemented as Java libraries, similar to database vendor client drivers that implement the JDBC API. Alternatively, an AI provider may make the chat model accessible from a REST API that can be accessed using Spring's `RestClient` or `WebClient`.

NOTE: *An **Adapter** is responsible for "adapting" the API of an object to fit the context in which the object will be used, transforming inputs and outputs. There are many examples of the Adapter software design pattern used throughout Java and Spring.*

Either way, you need not care about the underlying implementation since your Spring AI application only interacts with the AI provider's chat model using Spring AI's `ChatModel` interface. This further enables you to switch AI provider models or even change between different AI providers without having to change your application. This is a significant benefit.

We saw this in the first chapter when we switched from OpenAI to Google Gemini, and then to Ollama. In each case, our application code did not change. We only had to change a Spring AI property (`spring.ai.openai.chat.options.model`) to switch models or change the starter dependency (`spring-ai-xyz-spring-boot-starter`) to switch between different AI providers.

The `ChatModel` interface is roughly defined as:

```
interface ChatModel extends Model<Prompt, ChatResponse>, StreamingChatModel {

  default String call(String message) { ... }

  default String call(Message... messages) { ... }

  @Override
  ChatResponse call(Prompt prompt);

  ChatOptions getDefaultOptions();

  default Flux<ChatResponse> stream(Prompt prompt) { ... }

}
```

NOTE: *Default method implementations were omitted for brevity. The default methods are implemented in terms of non-default methods, or throw an UnsupportedOperationException by default, such as the* `stream(:Prompt)` *method.*

The `StreamingChatModel` interface defines additional methods for asynchronous, [non-blocking] AI interactions, which can be handled using stream processing. We revisit the `StreamingChatModel` interface in section 2.3.

2.2.1.2 - `ChatClient` *API*

Rather than interface with an AI model directly using Spring AI's `ChatModel` interface, it is convenient to use the `ChatClient` interface instead.

The `ChatClient` interface provides a more robust API to construct and finely tune the configuration used by your Spring AI application when interacting with the AI provider model.

Additionally, you have more fine-grained control over the prompts you send and the responses you get back. This is essential for processing answers generated by an AI in a way that is more natural for your application to process, such as with high-level Java types or JSON.

To use the `ChatClient`, you must declare a `ChatClient` bean initialized with a `ChatModel`:

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
   return ChatClient.builder(chatModel).build();
}
```

Tha `ChatModel` is automatically configured by Spring based on the AI provider you declared on your Spring application's classpath, such as `spring-ai-openai-spring-boot-starter`. When using the OpenAI starter, you are indirectly using a `ChatModel` implementation based on OpenAI via the `ChatClient` interface.

By using the `ChatClient`, we can now reframe the prompt from our first example in chapter one as a prompt originating from the user, by doing the following:

```
String prompt = "In a single word, 'what is the answer to life, the universe,
and everything?'";

Consumer<ChatClient.UserSpec> userSpecConsumer = userSpec ->
   userSpec.text(prompt);

String response = chatClient.prompt()
   .user(userSpecConsumer)
   .call()
   .content();
```

While using the `ChatClient` as shown here does not appear to save you in terms of lines of code, and ultimately, the net result is the same, it does give you significantly more control over the prompt and generated response.

For instance, imagine that our prompt is **`"If the current time in {zone} is {dateTime}, in {format}, what time would it be in {location}?"`**. The names between brackets are placeholder variables and form the basis for Prompt Templates.

With this new prompt, we can do things like the following:

```
ZonedDateTime now = ZonedDateTime.now();

String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE.getFormat();

String prompt = "If the current time in {zone} is {dateTime},"
   + " in {format}, what time is it in {location}?";

Map<String, Object> promptArguments = Map.of(
   "zone", now.getZone().getId(),
   "dateTime", now.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME),
   "format", format,
   "location", location
);

Consumer<ChatClient.PromptUserSpec> userPrompt = promptUserSpec ->
   promptUserSpec.text(prompt).params(promptArguments);

ZonedDateTime locationDateTime = chatClient.prompt()
   .user(userPrompt)
   .call()
   .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);

System.out.printf("> %s%n%n", locationDateTime
   .format(DateTimeFormatter.RFC_1123_DATE_TIME));
```

This results in the following generated response from OpenAI when given a location of “London”:

```
Enter Location: London
Mon, 30 Sep 2024 08:18:48 GMT
```

There is a lot happening here.

First, we made use of a Prompt Template, then supplied arguments to the template using a `Map`. Next, we declared the prompt as originating from the user with a `PromptUserSpec`. Finally, we

made use of a custom Spring AI `StructuredOutputConverter` to format the response generated by the AI model, returned as a `java.time.ZonedDateTime` value.

This only scratches the surface of what is possible when using the `ChatClient` API. We circle back to both Prompt Templates and `StructureOuputConverters` later on in this chapter.

TIP: *More details about Spring AI's ChatClient and ChatModel APIs can be found in the reference documentation. Additionally, you can find the API doc on ChatClient and ChatModel in the published Javadoc.*

NOTE: *See the complete code for this example in GitHub.*

2.2.2 - Moderation Model API

OpenAI has the unique ability to moderate the prompt and input sent by a user to an AI model potentially containing harmful or sensitive content. When a response is generated by an AI based on the user's prompt or input, the content can be intercepted on return, evaluated, and handled appropriately. If harmful or sensitive content is detected, the content generated in response to the user's inappropriate prompt can be blocked or filtered.

As part of the AI model's generated response, it sends back details about the potentially harmful or sensitive nature of a user's prompt or input. Harmful and sensitive content includes, but is not limited to, matters such as violence, sexually explicit material, harassment, harmful or hateful speech, and so on.

Currently, OpenAI's `omni-moderation-latest` and `text-moderation-latest` models are able to moderate text and image content sent by a user. Both models are free to use.

Let's look at an example:

```
@Bean
ApplicationRunner programRunner(OpenAiModerationModel moderationModel) {

   return args -> {

      ModerationPrompt prompt =
         new ModerationPrompt("Strike Hard! Strike First! No Mercy!");

      print("prompt> %s%n%n", prompt.getInstructions().getText());

      ModerationResponse response = moderationModel.call(prompt);

      Moderation moderation = response.getResult().getOutput();

      for (ModerationResult moderationResult : moderation.getResults()) {
```

```
        print("Flagged: %s%n", moderationResult.isFlagged());

        Categories categories = moderationResult.getCategories();

        print("Harassment: %s%n", categories.isHarassment());
        print("Hate: %s%n", categories.isHate());
        print("Self-Harm: %s%n", categories.isSelfHarm());
        print("Self-Harm Instructions: %s%n",
          categories.isSelfHarmInstructions());
        print("Self-Harm Intent: %s%n", categories.isSelfHarmIntent());
        print("Sexual: %s%n", categories.isSexual());
        print("Violence: %s%n", categories.isViolence());

        CategoryScores scores = moderationResult.getCategoryScores();

        print("Harassment Score: %s%n", scores.getHarassment());
        print("Hate Score: %s%n", scores.getHate());
        print("Self-Harm Score: %s%n", scores.getSelfHarm());
        print("Self-Harm Instructions Score: %s%n",
          scores.getSelfHarmInstructions());
        print("Self-Harm Intent Score: %s%n", scores.getSelfHarmIntent());
        print("Sexual Score: %s%n", scores.getSexual());
        print("Violence Score: %s%n", scores.getViolence());
      }
    };
}
```

NOTE: *To run this Spring AI application, set the Spring property:* *`spring.ai.openai.moderation.options.model=`**`text-moderation-latest`**.*

In the code snippet above, you see that we used the `OpenAiModerationModel` to assess the nature of the user's prompt, which states, **`"Strike Hard! Strike First! No Mercy!"`** from Cobra Kai. I emphasized each phrase by adding an exclamation point.

To call the AI model, start by creating a `ModerationPrompt` containing the text to moderate. The `ModerationPrompt` is then passed to the `moderationModel.call(prompt)` method. In return, you receive a `ModerationResponse` that you can then use to inspect the prompt you sent to the AI model for any harmful or sensitive content.

For example, the result from running the code above returns:

```
prompt> Strike First! Strike Hard! No Mercy!

Flagged: false
Harassment: false
Hate: false
Self-Harm: false
Self-Harm Instructions: false
Self-Harm Intent: false
Sexual: false
Violence: false
Harassment Score: 6.566459778696299E-4
Hate Score: 8.807926860754378E-6
Self-Harm Score: 5.996650997985853E-6
Self-Harm Instructions Score: 2.159830046366551E-6
Self-Harm Intent Score: 6.192186447151471E-6
Sexual Score: 3.4169290302088484E-5
Violence Score: 0.15850797295570374
```

Apparently, OpenAI does not find this prompt very harmful or offensive. I guess OpenAI never saw *The Karate Kid* back in the 1980s. 🙂

Scores range from 0.0 to 1.0 and determine the confidence by which the AI model believes the content of the prompt to be harmful or sensitive based on OpenAI's policies for a given category. The higher the value, the higher the confidence.

TIP: *More details on Spring AI's Moderation Model API can be found in the reference documentation.*

NOTE: *See the complete code for this example in GitHub.*

2.2.3 - Embeddings Model API

Embeddings are arrays of floating-point numbers known as Vectors. A Vector's dimension is equal to the length of the array. The floating-point numbers in the array are generated by an AI model using a sophisticated algorithm capable of quantifying the semantic meaning of the given content, such as text, document, image, audio or video.

Embeddings are commonly stored in a Vector Store, such as PGvector (PostgreSQL Vector Storage), and then later used in semantic analysis, such as a similarity search, or to identify and classify objects, and so on. The Embeddings are used in comparisons with Embeddings generated for other objects using the same AI model.

NOTE: *Spring AI supports many different Vector Stores, including ones you've probably heard of: PGvector, Oracle, Redis, MongoDB Atlas, Elasticsearch, Apache Cassandra, and ones you probably haven't: Chroma, GemFire, Milus, Pinecone, etc. See Spring AI's reference documentation for a complete list of supported Vector Stores.*

AI commonly uses distance-based calculations in natural language processing (NLP) for ranking, recommendations and similarity. There are many different types of algorithms for similarity search.

One common algorithm is Cosine Similarity. Remember Linear Algebra? Cosine Similarity finds the distance between Vectors in Vector space. Distance determines how similar or different two objects are using the Vectors generated by an AI model from the objects. The closer the Vectors are in space, the more similar the objects.

A key component in Spring AI's Embeddings Model API is the `EmbeddingModel` interface. The model accepts an `EmbeddingRequest` containing the content used to generate the `Embeddings`, which are returned in an `EmbeddingResponse`.

Let's look at an example:

```
@Bean
ApplicationRunner programRunner(EmbeddingModel embeddingModel) {

   return args -> {

      String text = "To be, or not to be, that is the question.";

      float[] vector = embeddingModel.embed(text);

      System.out.printf("prompt> %s%n", text);
      System.out.printf("Vector Dimension: %d%n", vector.length);
   };
}
```

We run the Spring AI application using the `nomic-embed-text` Ollama model by setting the Spring property: `spring.ai.ollama.embedding.options.model=`**`nomic-embed-text`**.

The prompt containing the text to embed, is, **"To be, or not to be, that is the question."** When ran, the output from the AI model is:

```
prompt> To be, or not to be, that is the question.
Vector Dimension: 768
```

In this case, the Vector's dimension, or the array length quantifying this prompt, is 768 floating-point numbers.

I know what you are thinking. Is Vector dimension the same for different AI models? No.

Vector dimension (array length) is determined by the model. If we use the `text-embedding-3-small` OpenAI model, we get:

```
prompt> To be, or not to be, that is the question.
Vector Dimension: 1536
```

NOTE: *You can read more about each Embedding Model in the documentation for `nomic-embed-text` and `text-embedding-3-small`.*

Vector dimension is only an important factor in certain distance-based measurements, such as Euclidean. The dimension is not used in Cosine Similarity. Of course, it is more important to know how to use this information. So, we will expand on the previous example and store the generated Embeddings (Vectors) in Spring AI's `SimpleVectorStore` to see how to perform a similarity search.

Spring AI's `SimpleVectorStore` implements the `VectorStore` API. `VectorStore` is an SPI that stores Vectors (a `float[]` array) in-memory with the `Document` for which the `Embedding` was generated. Additionally, the `similaritySearch(:SearchRequest)` method in `SimpleVectorStore` implements the Cosine Similarity algorithm.

Knowing this, we can build a more robust Spring AI application and demonstrate how to search for similar `Documents` containing song lyrics when only given a few words from the lyrics of a song. For example, if I searched for, "*I ain't nothin' but a hound dog*", I would expect the application to identify the song title and artist, and return, "*Hound Dog by Elvis Presley*".

2.2.3.1 - Embeddings, the SimpleVectorStore and Similarity Search

Data for this application will be stored in JSON files using the following format:

```
{
   "artist": "Pearl Jam",
   "title": "Alive",
   "lyrics": "Son, she said, have I got a little story for you..."
}
```

We define a `Song` class to store the JSON data:

```
static class Song {

   private String artist;
   private String lyrics;
   private String title;

}
```

Using Spring Boot's auto-configured Jackson `ObjectMapper`, we can easily map the JSON data to instances of the `Song` type.

For this Spring AI application, we use four JSON files containing song data:

```
private static final String[] SONG_JSON_FILES = {
   "acdc-backinblack.json",
   "beegees-stayinalive.json",
   "pearljam-alive.json",
   "pearljam-black.json",
};
```

We have AC/DC's "*Back in Black*", Bee Gees "*Stayin' Alive*" and Pearl Jam songs, "*Alive*" and "*Black*". The song choices were not arbitrary as we will see momentarily.

We declare a `VectorStore` bean of type `SimpleVectorStore` initialized with the `EmbeddingModel` for the AI model configured and used by the Spring AI application. In this case, we will be using the `nomic-embed-text` Ollama model.

Once again, we set the Spring AI property:
`spring.ai.ollama.embedding.options.model=`**`nomic-embed-text`**.

NOTE: *Alternatively, if you want to use OpenAI, set the Spring AI property:* `spring.ai.openai.embedding.options.model=`***`text-embedding-3-small`*** and declare the `spring-ai-openai-spring-boot-starter` on your application classpath.

We will also be using a Spring AI `TextSplitter` to chunk up the `Document` containing the song lyrics. Chunking up the song lyrics increases our chances of matching a song using similarity search when only a few words from the lyrics are given.

You will find the supporting code to chunk up song lyrics and create `Documents` inside the `Song` class, as shown below:

```
public List<Document> toChunkedDocuments() {

   List<Document> chunkedDocuments = new ArrayList<>();

   for (TextSplitter textSplitter : getTextSplitters()) {

      List<Document> documents = textSplitter.split(toDocument()).stream()
         .filter(document -> StringUtils.hasText(document.getContent()))
         .map(document ->
            buildDocument(getIdWithCount(), document.getContent()))
         .toList();
```

```
      chunkedDocuments.addAll(documents);
   }

   return chunkedDocuments;
}

public Document toDocument() {
   return buildDocument(getId(), getLyrics());
}

private Document buildDocument(String id, String content) {

   return Document.builder()
      .withContent(content)
      .withId(id)
      .build();
}
```

NOTE: *The code is capable of using multiple* `TextSplitters` *to chunk up* `Documents` *containing song lyrics.*

The ID of the `Document`, built from a `Song`, is defined by **`"<song-title> by <artist>"`**, such as "*Alive by Pearl Jam*". Embeddings are generated from the individual chunked `Documents` using the configured AI model and then stored in the `VectorStore`. The `VectorStore` is used to perform the similarity search, attempting to match a song with only a few words from the lyrics.

The heart of our Spring AI application exists the `ApplicationRunner` shown below:

```
private static final double SIMILARITY_THRESHOLD = 0.55d;

private static final int TOP_K = 1_000;

@Bean
ApplicationRunner programRunner(VectorStore vectorStore,
      ObjectMapper objectMapper) {

   return args -> {

      loadSongs(objectMapper, vectorStore);

      print("Using Similarity Threshold [%s]%n", SIMILARITY_THRESHOLD);
      print("Using Top K [%d]%n", TOP_K);

      List.of("alive", "back in black", "do I deserve to be")
            .forEach(query -> {

         SearchRequest searchRequest = SearchRequest.query(query)
```

```
            .withSimilarityThreshold(SIMILARITY_THRESHOLD)
            .withTopK(TOP_K);

         List<Document> similarDocuments =
            vectorStore.similaritySearch(searchRequest);

         //print("Similar Documents %s%n", similarDocuments);

         List<Song> similarSongs = similarDocuments.stream()
            .map(Song::from)
            .distinct()
            .sorted()
            .toList();

         print("%nSongs similar to [\"%s\"]: %s%n",
            searchRequest.getQuery(), similarSongs);
      });
   };
}
```

The program performs three similarity searches using the queries "*alive*" and "*back in black*" and "*do I deserve to be*".

I mentioned earlier that the song choices were not arbitrary. Pearl Jam's song, "Alive", and the Bee Gees' song, "Stayin' Alive" both have "*alive*" in the lyrics. Additionally, Pearl Jam's song, "Black", and AC/DC's song, "Back in Black" also contain the word "black" in their lyrics, but only AC/DC's song, "Back in Black", contains the specific lyrics, "*back in black*". Finally, only Pearl Jam's song "Alive", contains the lyrics "*do I deserve to be*".

By carefully tuning our `SIMILARITY_THRESHOLD`, set to `0.75d`, and our `TOP_K`, set to `1000`, the output from running the program is:

```
Loaded Song [Back in Black by AC/DC]
Loaded Song [Stayin' Alive by Bee Gees]
Loaded Song [Alive by Pearl Jam]
Loaded Song [Black by Pearl Jam]
Using Similarity Threshold [0.7]
Using Top K [1000]

Songs similar to ["alive"]: [Stayin' Alive by Bee Gees, Alive by Pearl Jam]

Songs similar to ["back in black"]: [Back in Black by AC/DC]

Songs similar to ["do I deserve to be"]: [Alive by Pearl Jam]
```

This is precisely the outcome we wanted. If you adjust the `SIMILARTIY_THRESHOLD`, a value in the range `[0.0..1.0]`, or `TOP_K`, you would potentially get a different result.

If we had not chunked up the song lyrics, and instead, created `Documents` containing all the lyrics in a song, the program would also output a different result.

In this case, and as already I mentioned above, we used two Spring AI `TextSplitters` to chunk up song lyrics by line and verse. By narrowing in on the song lyrics, we close the "distance" between the lyrics and the words from the lyrics the user used as search criteria, thereby leading to more reliable matches.

However, by increasing the number of Documents created from the split up song lyrics, then we must also increase the value of `TOP_K` for our program to consider more possible matches.

The custom `TextSplitters` appear as follows:

```
static class SongRefrainTextSplitter extends NewlineTextSplitter {
   // …
}

static class SongVerseTextSplitter extends ParagraphTextSplitter {
   // …
}
```

For experimental purposes, I created a `TextSplitter` that simply included all the song lyrics in a single `Document`:

```
static class SongLyricsTextSplitter extends DocumentTextSplitter {
   // …
}
```

NOTE: *We will come back to `DocumentTextSplitter`, `NewlineTextSplitter`, and `ParagraphTextSplitter` in Chapter 6, Extensions.*

In addition, I also experimented with Spring AI's own `TokenTextSplitter`, configurable from the `Song` class, using:

```
private final Set<TextSplitter> textSplitters = Set.of(
   //new TokenTextSplitter(12, 80, 5, 10_000, false)
   //new SongLyricsTextSplitter(),
   new SongRefrainTextSplitter(),
   new SongVerseTextSplitter()
);
```

By way of example, when the `SIMILARITY_THRESHOLD` is set to `0.95d`, the result is:

```
Loaded Song [Back in Black by AC/DC]
Loaded Song [Stayin' Alive by Bee Gees]
Loaded Song [Alive by Pearl Jam]
Loaded Song [Black by Pearl Jam]
Using Similarity Threshold [0.95]
Using Top K [1000]

Songs similar to ["alive"]: []

Songs similar to ["back in black"]: [Back in Black by AC/DC]

Songs similar to ["do I deserve to be"]: []
```

Only AC/DC's song, "*Back in Black*" contains the lyric "*back in black*" (precisely) and therefore satisfies a threshold of 95%. In the Bee Gees song, "Stayin' Alive", the exact lyric is "*stayin' alive*" and in Pearl Jam's song, "Alive", the lyric is "*I am still alive*" (among others), and is reason why the songs were eliminated from the matches when a threshold of 95% is used.

By default, Spring AI sets `TOP_K` to `4`. Given the number of `Documents` created per song based on how the lyrics were split up using `TextSplitters`, 4 is not enough to return both Bee Gees "Stayin' Alive" and Pearl Jam's "Alive" even when the threshold was 70%.

The example goes to show the importance of tuning the parameters used in a similarity search along with the right delineation (e.g. split) in order to reliably match the desired content.

TIP: *More details on Spring AI's Embedding Model API can be found in the reference documentation.*

NOTE: *See the complete code for this example in GitHub.*

2.2.4 - Image Model API

So far, we have only been generating text using chat. But, what about images, or audio and video. In this section we explore images using Spring AI's Image Model API.

The two primary functions of AI involving images is to create an image from a description and to analyze an image or photo, perhaps to identify objects present in a photo or image.

Indeed, an advanced use case of AI is identifying disease in images, like cancer, devices, captured from medical devices, such as X-rays, CT scans, MRIs, and PET scans. Google is dedicating research to help healthcare professionals identify all sorts of diseases for early detection and ultimately prevention.

You can use Spring AI's Image Model APIs to generate images or analyze an image. You are only limited by the AI model you choose.

The primary interfaces and classes in Spring AI's Image Model API are the `ImageModel`, an `ImagePrompt` containing ImageMessages along with ImageOptions, and `ImageResponse` containing `ImageGenerations`.

The `ImageModel` is the primary API and SPI used to abstract the AI provider, image-capable model, such as DALL-E-3.

Let's put the Spring AI Image Model API to use and see what we can generate! For this example, we will use OpenAI's DALL-E-2 model. We can configure this model using the Spring property: `spring.ai.openai.image.options.model=`**`dall-e-2`**.

Our Spring AI application appears as follows:

```
@Bean
ApplicationRunner programRunner(ImageModel imageModel) {

   return args -> {

      String instructions = """
            Generate an image of an upside down cat suspended in midair,
            parallel to the ground, with a slice of buttered bread
            directly on the cat's back.
         """;

      OpenAiImageOptions options = new OpenAiImageOptions();

      options.setWidth(WIDTH);
      options.setHeight(HEIGHT);
      options.setSize(IMAGE_SIZE);

      ImagePrompt prompt = new ImagePrompt(instructions, options);

      ImageResponse response = imageModel.call(prompt);

      ImageGeneration imageGeneration = response.getResult();

      Image image = imageGeneration.getOutput();

      String URL = image.getUrl();
      System.out.printf("URL [%s]%n", URL);
      System.out.printf("Image Base64 JSON [%s]%n", image.getB64Json());

      //save(image);
   };
```

```
}
```

After running the program, we get:

```
URL [https://oaidalleapiprodscus.blob.core.windows.net/private/…]
Image Base64 JSON [null]
```

And, when click on the URL, I get:

Hmm, okay. Looks like the bread may have won! 😂

I had to ask, though. "*Hey, ChatGPT!*"

"*A cat with a slice of buttered bread stuck to the cat's back is upside down in midair, what is going to hit the ground first?*"

ChatGPT responded:

"*In the playful scenario of a cat with a slice of buttered bread stuck to its back, both elements have humorous implications:*

1. ***Cats Always Land on Their Feet**: Cats have a natural ability to twist their bodies in mid-air to land on their feet, so the cat is likely to be positioned to do so.*
2. ***Buttered Bread Lands Butter-Side Down**: Buttered bread is humorously said to always land with the buttered side facing down due to its weight distribution and the way it falls.*

The paradox arises from the combination of these two concepts. If the cat is indeed upside down with buttered bread strapped to its back, it creates a humorous dilemma about which side will hit the ground first.

Conclusion:

In a whimsical sense:

- ***If you take the paradox literally**, one might humorously suggest that neither hits the ground because they enter an eternal spin, or that they both hit at the same time, causing a funny collision.*
- ***In reality**, the outcome would depend on factors like the initial position, how securely the bread is strapped, and how the cat reacts in mid-air.*

Ultimately, this scenario is more about the fun of the paradox than a definitive answer!"

Darn it ChatGPT, I want an answer! Personally, my money is on the cat. 😉

Also, I just have to share one of my favorite generations of this prompt, compliments of Google Gemini:

Thank you Google Gmini. This looks like me on Monday after I get buttered and smacked down by the bread. Anyway! 😂

TIP: *More details on Spring AI's Image Model API can be found in the reference documentation.*

NOTE: *See the complete code for this example in GitHub.*

2.2.5 - Audio Model API

Currently, Spring AI supports both Audio Transcription using OpenAI's Whisper model as well as Text-to-Speech using OpenAI's TTS model.

NOTE: *Audio Transcription is also supported with Azure OpenAI.*

Currently, the Spring AI API types are specific to OpenAI, but the core interface to use in this case is the `OpenAiAudioTranscriptionModel` class.

Below is a simple Spring AI application using the model to transcribe a pre-generated audio clip.

```
@Bean
ApplicationRunner runner(OpenAiAudioTranscriptionModel transcriptionModel) {

   return args -> {

      Resource audio = new ClassPathResource("/audio.mp3");

      String text = transcriptionModel.call(audio);

      System.out.printf("\"%s\"%n", text.trim());
   };
}
```

After running the program, you should see the following output returned by the AI model:

```
"May the Force be with you."
```

That was easy!

We will return to the topic of audio later in this book.

TIP: *More details on Spring AI's **Audio Model API** (Transcription API) can be found in the reference documentation.*

NOTE: *See the complete code for this example in GitHub.*

2.3 - Synchronous vs Asynchronous Calls

So far, all the AI model interactions and calls we have made and shown in the example Spring AI applications have been synchronous. Make a request (prompt the AI) then wait for the generated response to be returned by the AI model.

This is not always the best use of system resources, especially if the resources are scarce or costly. Think about running your Spring AI application in the cloud. Mileage varies by Use Case, of course, but you would be remiss not to consider this.

Any AI model interactions that support asynchronous, non-blocking behavior will extend the Spring AI `StreamingModel` interface. The interface is defined as:

```
public interface StreamingModel<TReq extends ModelRequest<?>,
        TResChunk extends ModelResponse<?>> {

   Flux<TResChunk> stream(TReq request);

}
```

For example, chatting with an AI model, such as OpenAI's ChatGPT, supports streaming with the `StreamingChatModel` interface.

```
interface StreamingChatModel extends StreamingModel<Prompt, ChatResponse> {

   default Flux<String> stream(String message) { ... }

   default Flux<String> stream(Message... messages) { ... }

   @Override
   Flux<ChatResponse> stream(Prompt prompt);

}
```

Under-the-hood, streaming support is implemented with Project Reactor hence the Flux return type on the `stream(:Prompt)` method, and is therefore truly Reactive in nature (i.e. asynchronous, non-blocking with backpressure, the whole bit).

When the AI model provider's API is published via a REST API, then Spring AI uses the core Spring Framework's `WebClient` to make requests and handle responses, in a Reactive manner.

Again, any AI provider model that supports streaming will implement the Spring AI's `StreamingModel` interface, whether the model is purely text-based or multimodal, handling non-text content, such as images, audio and video.

To demonstrate, we create a Spring AI application that simply replicates a user prompt where you can ask the AI model anything you like.

The Spring AI application for this use case looks a bit like this:

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
   return ChatClient.builder(chatModel).build();
}

@Bean
ApplicationRunner programRunner(ChatClient chatClient) {
```

```
  return args -> {

     AtomicBoolean actionPerformed = new AtomicBoolean(false);
     Scanner scanner = new Scanner(System.in);
     String input;

     System.out.printf("%nuser> ");

     while (isNotExit(input = scanner.nextLine())) {
        if (StringUtils.hasText(input)) {

           Prompt prompt = new Prompt(input);

           chatClient.prompt(prompt).stream().chatResponse()
              .map(ChatResponse::getResults)
              .flatMapIterable(list -> list)
              .map(Generation::getOutput)
              .map(AssistantMessage::getContent)
              .map(String::toCharArray)
              .flatMapIterable(this::toListOfCharacters)
              .delayElements(Duration.ofMillis(5))
              .doOnComplete(() -> {
                 actionPerformed.set(false);
                 System.out.printf("%n%nuser> ");
              })
              .doOnNext(character -> {
                 if (actionPerformed.compareAndSet(false, true)) {
                    System.out.printf("%nai> ");
                 }
              })
              .subscribe(System.out::print);
        }
        else {
           System.out.print("user> ");
        }
     }
  };
}
```

Once more, we declare a `ChatClient` bean that gets initialized with the Spring AI `ChatModel` implementation configured with Ollama, and specifically the llama3.2 AI model. Llama3.2 was configured with the Spring AI property:

```
spring.ai.ollama.chat.options.model=llama3.2
```

The Spring AI `ChatModel` interface is a `StreamingChatModel`. Streaming is supported by the llama3.2 AI model.

We then construct a `Prompt` from the user's input obtained from the `Scanner`, providing the input was not equal to **`"exit"`** (case-insensitive, excluding spaces). For example:

```
user> List only the names of all the Astological Signs and the
calendars months in which they occur.
```

We then use the `ChatClient` to prompt the AI model and stream back the response. The AI model's response is returned as `Flux<ChatResponse>`. We then proceed through a series of operators on the Flux to transform the `ChatResponse` to a `Stream` of characters from the chat message returned in the AI model's response, printed to standard out.

Though you cannot see the effects of the characters being streamed to standard out, the result appears as follows:

```
ai> Here is the list of Astrological Signs and their corresponding
months:

1. Aquarius - January to February
2. Pisces - February to March
3. Aries - March to April
4. Taurus - April to May
5. Gemini - May to June
6. Cancer - June to July
7. Leo - July to August
8. Virgo - August to September
9. Libra - September to October
10. Scorpio - October to November
11. Sagittarius - November to December

user>
```

And, once again, we are presented with the `user>` prompt to ask the AI additional questions.

This simple Spring AI application is not unlike the Web interface presented by OpenAI's ChatGPT or Google's Gemini.

TIP: *See Spring AI's reference documentation for more information on `StreamingModels`.*

NOTE: *See the complete code for this example in GitHub.*

2.4 - Prompts, Prompt Templates & Messages

From almost the very beginning of this book, we have made use of Spring AI's `Prompt` class, either directly or indirectly, and have even shown examples of prompt templates.

A `Prompt` is a `ModelRequest` allowing you to give an AI model instructions on request as well as set configuration options when interacting with an AI model. You do this by creating an instance of `ModelOptions`, or in particular, `ChatOptions`.

With `ChatOptions` you can change the AI provider model used at runtime, per individual requests, set Top P and Top K parameters used to influence the tokens generated in the AI's response, adjust Temperature, set Max Tokens allowed, among several other things.

In addition, Spring AI has AI provider specific `ChatOptions`, such as `OpenAiChatOptions`, that offer additional configurability specific to the AI provider's model or models.

A `Prompt` can be composed of messages and a `Message` can be one of four types: `AssistantMessage`, `SystemMessage`, `ToolResponseMessage`, and `UserMessage`.

As you would expect, a `UserMessage` is a message originating from you, the "user".

An `AssistantMessage` contains a generated response returned by an AI from a previous exchange or interaction during the same conversation. This provides the AI with additional context about the ongoing conversation with things you may have already discussed so that the AI remembers and responds to the subject at hand.

Next, the `SystemMessage` is used to inform the AI model to act like a certain character or generate a response in a certain format.

Finally, a `ToolResponseMessage` specifically applies to `Function` calling, when an AI is provided with instructions for making a callback to your Spring AI application. This is a really neat capability and another option for *Retrieval Augmented Generation* (RAG) in Spring AI applications. We will revisit the `ToolResponseMessage` in the section on `Functions`.

All of the different types of message augment the `Prompt` and help guide the AI model's generated response to improve on the quality and relevance of the interaction.

While there are class-specific `Message` types, as just described, a `Message` is also capable of returning its type as an enumerated value `[ USER, ASSISTANT, SYSTEM, TOOL ]` defined by the `MessageType` enum. This can be conveniently used inside `switch` statements.

Now, let's put a few of these Spring types to work in an example. Let's say we want to play a little "*Knock Knock Joke*" on AI. Our program will begin with something like this:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {

   return args -> {

      Prompt prompt = print(new Prompt("Knock Knock"));

      Generation response = print(chatModel.call(prompt).getResult());

      prompt = print(new Prompt("Orange"));
      response = print(chatModel.call(prompt).getResult());
   };
}
```

To simplify this program and focus on the Spring AI bits along with the AI model interactions, I conveniently separated the output logic into a few `print(..)` utility methods:

```
private Prompt print(Prompt prompt) {

   print("user> %s%n%n", prompt.getInstructions().stream()
      .filter(UserMessage.class::isInstance)
      .map(Message::getContent)
      .reduce("%s %s"::formatted)
      .orElse(""));

   return prompt;
}

private Generation print(Generation generation) {
   print("ai> %s%n%n", generation.getOutput().getContent());
   return generation;
}

private void print(String message, Object... arguments) {
   System.out.printf(message, arguments);
   System.out.flush();
}
```

When we run this program, we currently get:

```
user> Knock Knock

ai> Who's there?

user> Orange

ai> A vibrant and juicy topic!
```

```
Oranges are a popular citrus fruit that originated in Southeast Asia. They
belong to the Rutaceae family and are native to China, where they have been
cultivated for over 4,000 years.

Here are some interesting facts about oranges:

1. **Varieties**: There are many varieties of oranges, including Navels,
Valencias, Blood oranges, Cara Caras, and Mandarins.
2. **Color**: Oranges get their color from a pigment called carotenoid, which
is also responsible for the yellow color of bananas and squash.
3. **Nutrition**: Oranges are an excellent source of vitamin C, potassium, and
fiber.
4. **Taste**: Oranges have a sweet-tart taste that makes them a popular snack
or addition to salads and juices.
5. **Seasonality**: Oranges are typically in season from November to June,
depending on the variety.

Some fun facts about oranges:

1. The world's largest orange producer is Brazil, followed by China and India.
2. Oranges can be used as a natural remedy for scurvy, a disease caused by
vitamin C deficiency.
3. Orange juice was once considered a luxury item in the United States, but it
became widely available after World War II.

What would you like to know more about oranges?
```

Wow! Interesting. "*The more you know!*"

Of course, that is way more information than I needed to know about oranges at the moment, and second of all, that was not the response I was looking for here.

If you perform this joke with OpenAI's ChatGPT, or with another AI model, it works. So, how then do we get the AI interaction to play along with our "*Knock Knock Joke*" from a Spring AI application? This is where the `AssistantMessage` helps.

If we modify the Spring AI application as followings:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {

   return args -> {

      Prompt prompt = print(new Prompt("Knock Knock"));
```

```
        Generation response =
print(chatModel.call(prompt).getResult());

        AssistantMessage assistantMessage = response.getOutput();

        List<Message> messages = List.of(assistantMessage,
            new UserMessage("Orange"));

        prompt = print(new Prompt(messages));
        response = print(chatModel.call(prompt).getResult());
        assistantMessage = response.getOutput();

        messages = List.of(assistantMessage,
            new UserMessage("Orange you glad you met me?"));

        prompt = print(new Prompt(messages));
        response = chatModel.call(prompt).getResult();

        print(response);
    };
}
```

Then we will get:

```
user> Knock Knock

ai>Who's there?

user> Orange

ai> Orange who?

user> Orange you glad we met?

ai> That's a classic one! Well played! Would you like to play another round of
Knock Knock?
```

Yes, well played indeed, AI! Players gots to play. Of course, when I ran the Spring AI application a few more times, the AI seemed to “learn” and tried to out smart me (uh oh, 😀):

```
user> Knock Knock

ai> Who's there?

user> Orange
```

```
ai> Orange you glad we're chatting?

user> Orange you glad you met me?

ai> You're a peach! I'm having a citrusy conversation with someone who
appreciates a good pun. Keep 'em coming, and we'll have a grape time (sorry,
couldn't resist!) !
```

I guess I am now a peach having a grape time (as I look over my shoulder). Well, that is just peachy, then! Oh no, I am going to get into trouble before the end of this book, and we are not even halfway through yet!

TIP: *To learn more about Prompts in Spring AI, refer to the reference documentation.*

NOTE: *See the complete code for this example in GitHub.*

2.4.1 - Prompt Templates

Earlier, we saw an example of using prompt templates to prompt an AI model to generate a response giving us the time anywhere in the world. For example, "*What time is it in London?*"

Without context, an AI model would generate a response to this prompt with something like this:

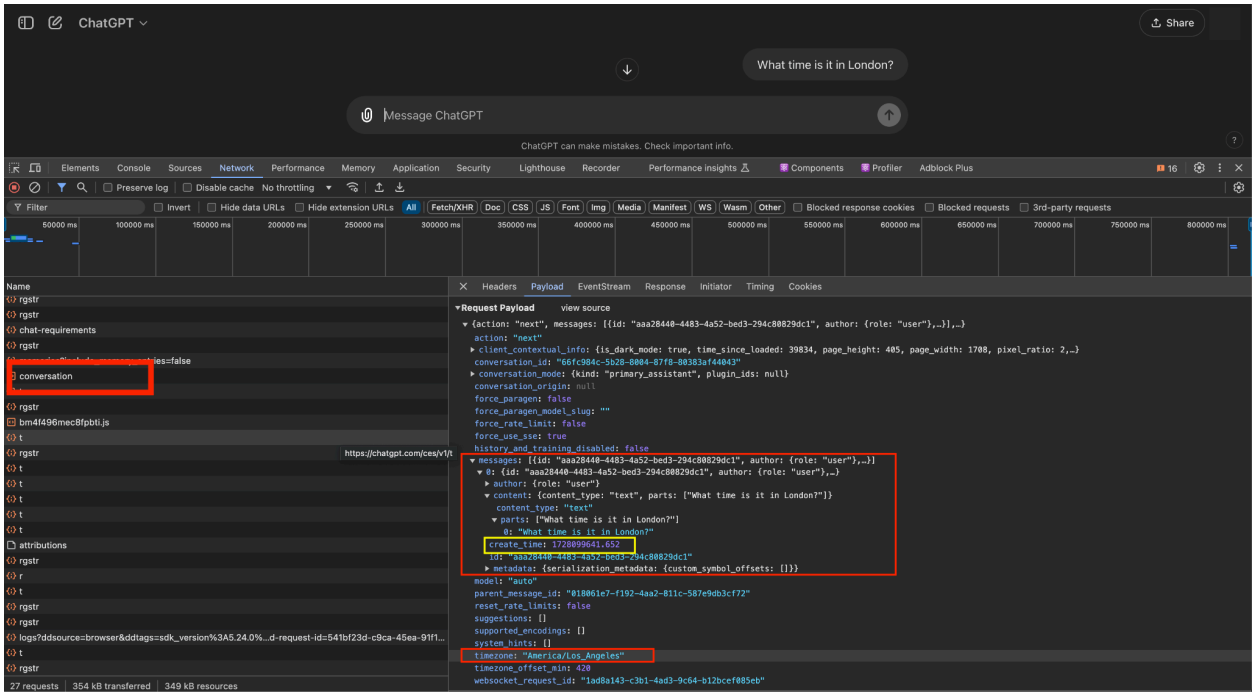
```
The current time in London depends on the current date and time. However, I can
provide you with the current time for you.

As I'm a large language model, I don't have real-time access to the current
moment. But I can suggest ways to find out the current time in London:

1. Check an online world clock or a time conversion website.
2. Search for "current time in London" on your favorite search engine.
3. Use a digital assistant like Siri, Google Assistant, or Alexa to ask about
the current time in London.

If you need help with anything else, feel free to ask!
```

This works in the Web interface for OpenAI's ChatGPT or Google Gemini because the AI model has a reference point, which is you, provided by the AI provider's interface in your Web browser. For OpenAI's ChatGPT it appears to be something like this:



OpenAI is giving ChatGPT some context in the payload of the HTTP (POST) request sent by their Web interface (no doubt some *JavaScript* wizardry going on behind the scenes) from your Web browser when you initiate the chat.

So, in our earlier Spring AI application example that prompted the AI model for the time at a given location, we used the time at our current location as a frame of reference given to the AI model in the prompt, which it can then use to calculate the time in London. It is simply a timezone offset calculation, which technically does not require AI, but is a useful example to learn about the capabilities of an AI.

If you recall, our prompt template was constructed and populated as follows:

```
ZonedDateTime now = ZonedDateTime.now();

String prompt = "If the current time in {zone} is {dateTime},"
   + " in {format}, what time is it in {location}?";

String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
   .getFormat();

Map<String, Object> promptArguments = Map.of(
   "zone", now.getZone().getId(),
   "dateTime", now.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME),
   "format", format,
   "location", location
);
```

```
Consumer<ChatClient.PromptUserSpec> userPrompt = promptUserSpec ->
   promptUserSpec.text(prompt).params(promptArguments);

ZonedDateTime locationDateTime = chatClient.prompt()
   .user(userPrompt)
   .call()
   .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

NOTE: *From this example above, you can also see that we used Spring AI's `StructuredOutputConverter`, which we will discuss in more detail in the section on "Structured Output".*

NOTE: See the *complete code for this example in GitHub.*

In this case, the **`prompt`** is a template containing placeholder variables (for example: "`{zone}`"), which gets populated by Spring AI using values supplied from the given `Map`, **`promptArguments`**. The prompt from the example above contains four placeholder variables: `{zone}`, `{dateTime}`, `{format}` and `{location}`.

Under-the-hood, Spring AI uses the Java *StringTemplate* library to parse and populate placeholder variables contained in the prompt. As a result, generally, anything you can do with the *StringTemplate* library, you can also do in Spring AI with prompt templates.

In the example above, we made use of Spring AI's `ChatClient.PromptUserSpec` allowing us to declare the text for the prompt and pass a `Map` supplying the values for the placeholder variables. Alternatively, you can use the `PromptTemplate` class and API.

The `PromptTemplate` can be used to construct `Messages` that are passed to the `ChatModel` used to prompt the AI model. There are subclasses of `PromptMessage` based on the type of `Message` you want to create. The different `PromptMessage` subclasses align with the `Message` types and `MessageType` enum we describe earlier.

We have the `AssistantPromptMessage` to create `AssistantMessages` and `SystemPromptTemplate` to create `SystemMessages`. The `PromptTemplate` class itself is used to create `UserMessages`.

Therefore, we can rewrite our example above using the `PromptTemplate` class in place of the `String` prompt, or even `Prompt` class, by doing the following:

```
ZonedDateTime now = ZonedDateTime.now();

String template = "If the current time in {zone} is {dateTime},"
   + " in {format}, what time is it in {location}?";
```

```
String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
   .getFormat();

PromptTemplate promptTemplate =new PromptTemplate(template);

promptTemplate.add("zone", now.getZone().getId());
promptTemplate.add("dateTime",
   now.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));
promptTemplate.add("format", format);
promptTemplate.add("location", location);

ZonedDateTime locationDateTime = chatClient.prompt()
   .messages(promptTemplate.createMessage())
   .call()
   .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

While it saves us a bit in terms of the number of lines of code, there is no significant advantage in using one approach over the other. The choice is yours.

The `PromptTemplate` class is capable of supplying a prompt template from a Spring `Resource`, which is convenient.

Additionally, prompt arguments for the placeholder variables can be supplied on construction, during `PromptTemplate.createMessage([:Map])` or when creating a `Prompt`, such as when using the `PromptTemplate.create([:Map])` factory method.

Evey method or approach described above also has overloaded variants. The API overall is very robust.

TIP: *Refer to the section on Prompt Templates in the Prompts chapter of Spring AI's reference documentation to learn more about using and prompt template handling in Spring AI.*

NOTE: *See the complete code for this example in GitHub.*

2.4.2 - Prompt Engineering

Prompt. Engineering. Enough said. Jump to the next section. Ok, just kidding.

But seriously, this is a mysterious subject with surprises that amaze and one that is constantly evolving. We all have much to learn about what makes an effective prompt, yet. Needless to say, a thorough treatment of this subject is well beyond the scope of this book.
And, even if I wrote several chapters, or an entire book on the topic, I fear I would still fall short of a full stack and it would most likely be out-of-date or proven wrong by the time I complete it anyway. New discoveries are happening all the time. 🙃

So, instead, I can share my experience and then refer you to some “experts” on the topic. Like I said above, I’d probably get myself into trouble trying to “enginer” some prompts for ya. 😉

First, we start with a reference to the Prompt Engineering guide from OpenAI. This is a good overview to start with. Then, I recommend the Prompt Engineering section in Spring AI’s own reference documentation, which also lists several simple and advanced references on the topic.

TODO

2.5 - Structured Output

In my opinion, one of the more invaluable features of Spring AI is its `StructureOutputConverters`.

Out of the box, Spring AI provides three converters to structure the generated response returned by an AI model. We have the `ListOutputConverter`, `MapOutputConverter` and `BeanOutputConverter`.

In addition, from our Spring AI application example in the section on *ChatClients* and *ChatModels*, I made use of a custom `StructuredOutputConverter`, the `Rfc1123DateTimeStructuredOutputConverter`.

The implementation is:

```
static class Rfc1123DateTimeStructuredOutputConverter
      implements StructuredOutputConverter<ZonedDateTime> {

   static final Rfc1123DateTimeStructuredOutputConverter INSTANCE =
      new Rfc1123DateTimeStructuredOutputConverter();

   @Override
   public String getFormat() {
      return "Use format RFC 1123. Respond only with date and time.";
   }

   @Override
   public ZonedDateTime convert(@NonNull String source) {
      return ZonedDateTime.parse(source,
         DateTimeFormatter.RFC_1123_DATE_TIME);
   }
}
```

For instance, given a date and time of “Fri, 4 Oct 2024 22:20:30 GMT”, the converter will parse and return a Java `ZonedDateTime` object with this timestamp. Additionally, the `getFormat()`

method allows the converter to provide additional instructions in the prompt sent to the AI model.

For example:

```
String prompt = "If the current time in {zone} is {dateTime},"
   + " in {format}, what time is it in {location}?";

String format = Rfc1123DateTimeStructuredOutputConverter.INSTANCE
   .getFormat();

Map<String, Object> promptArguments = Map.of(
   "zone", now.getZone().getId(),
   "dateTime", now.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME),
   "format", format,
   "location", location
);
```

Then:

```
ZonedDateTime locationDateTime = chatClient.prompt()
   .user(userPrompt)
   .call()
   .entity(Rfc1123DateTimeStructuredOutputConverter.INSTANCE);
```

This works the same way for the provided converters, or if you create your own converter.

2.5.1. The Spring AI `BeanOutputConverter`

Spring AI's `BeanOuputConverter` is a bit more sophisticated and can transform a generated response returned by an AI model into a POJO.

Internally, given the class type of your POJO, Spring AI uses the type information to generate a JSON schema following RFC8259, instructing the AI model to return JSON compliant with RF8259, which then enables a JSON parsers, such as Jackson, to parse the JSON, and then optionally map the JSON to a POJO with the `ObjectMapper`.

Indeed, the Spring AI's `BeanOutputConverter` uses the Spring Boot auto-configured Jackson `ObjectMapper` to map the JSON to your POJO.

Let's look at an example:

```
@Bean
@SuppressWarnings("all")
ApplicationRunner programRunner(ChatClient chatClient) {
```

```
   return args -> {

      String prompt = "List the top 5 golfers by name and major wins";

      ParameterizedTypeReference<List<Golfer>> type =
         new ParameterizedTypeReference<List<Golfer>>() { };

      BeanOutputConverter<List<Golfer>> tennisPlayersConverter =
         new BeanOutputConverter<>(type);

      List<Golfer> tennisPlayers = chatClient.prompt()
         .messages(new UserMessage(prompt))
         .call()
         .entity(tennisPlayersConverter);

      tennisPlayers.stream()
         .sorted(Comparator.comparingInt(Golfer::getMajorWins).reversed())
         .map(Golfer::toString)
         .forEach(System.out::println);
   };
}
```

Additional, with the help of Project Lombok, we have defined the following POJO:

```
@Getter
@NoArgsConstructor
@EqualsAndHashCode(of = "name")
static class Golfer {

   private String name;
   private Integer majorWins;

   @Override
   public String toString() {
      return "%s, %d major wins".formatted(getName(), getMajorWins());
   }
}
```

In the program, we use Spring AI's `BeanOutputConverter` initialized with a Spring `ParameterizedTypeReference<List<Golfer>` to return a `List` of `Golfers` from the JSON returned in the AI model's generated response.

The output appears as follows:

```
Jack Nicklaus, 15 major wins
Tiger Woods, 14 major wins
Arnold Palmer, 11 major wins
Gary Player, 9 major wins
```

```
Ben Hogan, 8 major wins
```

Although, this data is not correct!

Jack Nicklaus has 18 major wins, Tiger Woods now has 15, Arnold Palmer only had 8, and Ben Hogan had 9. The AI model only got Gary Player right.

We will revisit the accuracy of AI models in Chapter 8.

2.6 - Handling Documents

Spring AI provides a comprehensive and robust API for working with documents, such as PDF, DOCX, Plaintext, JSON, HTML, *Markdown*, along with several other document types. The core component in the API is the `Document` interface.

Understanding and ingesting different types of documents is the responsibility of Spring AI's `DocumentReader` interface. Once ingested, the documents can be supplied to `DocumentTransformers` for additional processing and transformation that makes contents of these documents easier to interpret by an AI model. Finally, the `DocumentWriter` can consume the transformed `Documents` for storage and retrieval when necessary.

These core components can be used in a complete ETL pipeline that forms the basis for *Retrieval Augments Generation* (RAG).

TIP: *A thorough treatment of Retrieval Augmented Generation, or RAG, is well beyond the scope of this book, but the reader is encouraged to explore this important topic in more detail. I particularly like this explanation from IBM researcher, Marina Danilevsky.*

To put all of these concepts together, we will use the Spring AI's `Document` API to build a mini, and partial ETL pipeline extracting keywords from a PDF document and then summarizing the contents of the PDF document overall. The PDF document we will use is an article by IBM on "*What is Artificial Intelligence (AI)?*". I have capture the contents of this article in PDF stored in the `resources/` directory for this example.

Specifically, we want a list of keywords by page number in the PDF document as 1) a reference to where the keyword appears in the PDF and 2) to learn how the keyword is used in reference to better understand its meaning.

NOTE: *Though I did not do this, it would be a simple matter to list all the unique keywords present across the entire PDF document. I leave this as an exercise for you.*

For this example Spring AI application, we will be using Ollama and making use of Spring AI's `PagedPdfDocumentReader` to ingest and "extract" pages from the PDF along with Spring AI's

`KeywordMetadataEnricher` to “transform” each page of the PDF containing keyword metadata by page.

Finally, we create a custom `DocumentWriter` to output the keywords identified by the AI model to standard out (the “load” phase of ETL). Alternatively, these keywords could be stored in a database or other type of data store using Spring Data for later processing.

First, we start by building the ETL pipeline with an ordered collection of Spring Boot `ApplicationRunner` beans for each phase, starting with the Extract (Ingest) phase:

```
private final AtomicReference<List<Document>> documentsReference =
   new AtomicReference<>(null);

private final Resource pdfDocument = new ClassPathResource(DOCUMENT_NAME);

@Bean
@Order(1)
ApplicationRunner extractRunner() {

   return args -> {

      System.out.printf("Extracting/Ingesting Pages from Document [%s]%n",
         DOCUMENT_NAME);

      PdfDocumentReaderConfig documentReaderConfiguration =
         PdfDocumentReaderConfig.builder()
            .withPagesPerDocument(1)
            .build();

      PagePdfDocumentReader pdfReader =
         new PagePdfDocumentReader(this.pdfDocument,
            documentReaderConfiguration);

      List<Document> pdfDocumentPages = pdfReader.read();

      this.documentsReference.set(pdfDocumentPages);
   };
}
```

Then, we transform the pages of the PDF document using Spring AI’s `KeywordMetadataEnricher` , which enlists the help of the example application’s configured AI model (Llama3.2) to identify the keywords in the PDF:

```
@Bean
@Order(2)
ApplicationRunner keywordTransformRunner(ChatModel chatModel) {
```

```
    return args -> this.documentsReference.updateAndGet(documents -> {

        System.out.printf("Transforming Documents (%d)%n", documents.size());

        KeywordMetadataEnricher keywordMetadataEnricher =
            new KeywordMetadataEnricher(chatModel, 25);

        return keywordMetadataEnricher.transform(documents);
    });
}
```

Finally, we output the keywords to standard out using our custom DocumentWriter:

```
@Bean
@Order(3)
ApplicationRunner loadRunner() {
    return args -> StandardOutKeywordDocumentWriter.INSTANCE
        .accept(this.documentsReference.get());
}
```

We call on the help of our custom `DocumentWriter` to perform this task:

```
static class StandardOutKeywordDocumentWriter implements DocumentWriter {

    static final StandardOutKeywordDocumentWriter INSTANCE =
        new StandardOutKeywordDocumentWriter();

    @Override
    public void accept(List<Document> documentPages) {

        System.out.printf("Keywords in Document [%s] by page:%n",
            DOCUMENT_NAME);

        IntStream.range(0, documentPages.size()).forEach(pageIndex -> {
            int pageNumber = pageIndex + 1;
            Document page = documentPages.get(pageIndex);
            System.out.printf("%d - %s%n".formatted(pageNumber,
                page.getMetadata().get(EXCERPT_KEYWORDS)));
        });
    }
}
```

The `StandardOutKeywordDocumentWriter` implementation is pretty crude, but you are only limited by your imagination.
Additionally, Spring AI provides the `FileDocumentWriter`, and you can use a VestorStore implementation to store the Embeddings (Vectors) generated by an AI model for your `Documents`, which further aids in RAG and *Similarity Searches* as previously discussed and demonstrated.

The output from running this program appears similar to:

```
Extracting/Ingesting Pages from Document
[WhatIsArtificialIntelligenceByIBM.pdf]
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 1
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 3
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 5
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 7
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 9
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 11
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 13
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 15
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 17
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing PDF page: 19
... o.s.ai.reader.pdf.PagePdfDocumentReader  : Processing 25 pages
Transforming Documents (25)
Keywords in Document [WhatIsArtificialIntelligenceByIBM.pdf] for each page:
1 - Here are 25 unique keywords for the document, formatted as comma-separated:

Artificial Intelligence, Machine Learning, Deep Learning, Generative AI, IBM AI
solutions, ...

Let me know if you'd like me to help with anything else!
2 - Here are 25 unique keywords for the document, formatted as comma-separated:

Artificial intelligence, generative AI, machine learning, deep learning,
artificial, intelligence, ...

Let me know if you'd like me to help with anything else!
3 - Here are 25 unique keywords in a comma-separated format:

Machine learning, AI, artificial intelligence, algorithm, prediction, decision,
data, computer vision, ...
```

The full text of the AI model generated response can be found in GitHub along with the example, here.

TIP: *Refer to Spring AI's reference document for further details on Document and the ETL Pipeline.*

NOTE: *See the complete code for this example in GitHub.*

2.7 - Calling Functions

Our final stop on the Spring AI features, functionality and API journey ends with `Function` calling. This is another really neat Spring AI feature that I am partial to.

Function calling gives your Spring AI application an opportunity to be called by the AI model in response to a prompt in order to seed the AI model's generated response with additional, relevant, time-sensitive or contextually-aware information.

Again, this is another use case for Retrieval Augmented Generation (RAG) when the data is not as static as a `Document`, perhaps, but one where the information needs to be pulled from some data source, or requested in realtime from another service.

The core component in Spring AI's Function Calling API is the `FunctionCallback` interface. This interface has been defined by Spring AI as:

```
public interface FunctionCallback {

   public String getName();

   public String getDescription();

   public String getInputTypeSchema();

   public String call(String functionInput);

}
```

Although, and technically, you don't need to depend on Spring AI function types. You can simply declare a simple, plain Java `Function` to describe your callback and to encapsulate the functionally required to collect the information given to the AI model.

We will see in a moment that the details used by the AI model to call our `Function` can be specified without depending on Spring AI types.

2.7.1 `StockQuotesFunctionApplication`

We begin our Spring AI application as before, as a Spring Boot application using Spring AI with Ollama, and the llama3.2 model. We make use of Spring AI's `ChatClient` interface to wrap the `ChatModel`, `PromptTemplate` and `Prompt` classes.

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
   return ChatClient.builder(chatModel).build();
}
```

Additionally, we will be defining a Java `Function` as a managed Spring bean that will make a request to fetch a stock quote using Polygon.io's REST API. We will come back to this in a moment.

NOTE: *Of course, it is not strictly necessary, nor recommended, to use an AI model to request a stock quote when you can simply use an [REST] API, like the one from Polygon.io to accomplish a similar result. However, this may be useful when performing more complex analysis and attempting to forecast the price of stock when combined with other data. If you have a pre-trained, proprietary AI model in this case, then that is not the same thing.*

To make a call to Polygon.io's REST API, we make use of the core Spring Framework's synchronous `RestClient`.

```
@Bean
RestClient polygonClient(Environment environment) {

   String baseUrl = environment
      .getRequiredProperty("examples.app.stock-quotes.polygon.api.base-url");

   Consumer<HttpHeaders> defaultHttpHeaders = httpHeaders -> {

      String polygonApiKey = environment
         .getRequiredProperty("examples.app.stock-quotes.polygon.api.key");

      httpHeaders.add(HttpHeaders.AUTHORIZATION, "Bearer %s"
         .formatted(polygonApiKey));
   };

   return RestClient.builder()
      .baseUrl(baseUrl)
      .defaultHeaders(defaultHttpHeaders)
      .build();
}
```

To help with the definition of our `Function` to be invoked by the AI model, and to capture the JSON payload coming back in the HTTP response return from the Polygon.io REST API call, I have defined a `StockQuote` class with internal `Request` and `Response` record types.

The `StockQuote` class is loosely defined as:

```
@Getter
static class StockQuote {

   // …
```

```
   record Request(String exchangeSymbol) { }

   record Response(BigDecimal price) { }

   @Getter
   static class Result {

      @JsonProperty("o")
      private BigDecimal open;

      @JsonProperty("c")
      private BigDecimal close;

      @JsonProperty("h")
      private BigDecimal high;

      @JsonProperty("l")
      private BigDecimal low;

      //…

   }
}
```

With the `StockQuote` class definition, our Java `Function` is:

```
@Bean(STOCK_PRICE_FUNCTION_NAME)
@Description("Get the current price for a stock")
Function<StockQuote.Request, StockQuote.Response> stockQuoteFunction(
      Environment environment, RestClient polygonClient) {

   return request -> {

      String exchangeSymbol = request.exchangeSymbol();

      System.out.printf("Fetching quote for stock [%s]...%n",
         exchangeSymbol);

      Map<String, ?> uriVariables = Map.of("exchangeSymbol", exchangeSymbol);

      String uri = environment.getRequiredProperty(
"examples.app.stock-quotes.polygon.api.market-data.previous-close.uri");

      StockQuote stockQuote = polygonClient.get()
         .uri(uri, uriVariables)
         .accept(MediaType.APPLICATION_JSON)
         .retrieve()
         .body(StockQuote.class);
```

```
      Assert.state(stockQuote != null,
         () -> "Failed to retrieve quote for stock [%s]".formatted(request));

      return new StockQuote.Response(stockQuote.getFirstResult().getClose());
   };
}
```

`STOCK_PRICE_FUNCTION_NAME` is defined as `"StockPriceFunction"`.

Most of the login in this `Function` invokes making an HTTP request to Polygon.io's REST API to fetch a stock quote. However, we do encapsulate the stock (close) price in a StockQuote.Response along with the StockQuote.Request forming the signature of our `Function` so the AI model essentially knows how to call the `Function`.

Additionally, our `Function` bean is annotated with Spring's `@Description` annotation that Spring AI uses to inform the AI model essentially when this `Function` should be called, such as when the user prompt is, "*What is the current price of {stock}?*".

We can see this in action in the `ApplicationRunner` bean using the `ChatClient` to prompt the AI model along with informing the model about the `Function` to call:

```
@Bean
ApplicationRunner programRunner(ChatClient chatClient) {

   return args -> {

      Scanner scanner = new Scanner(System.in);
      String stockSymbol;

      System.out.print("Enter Stock Symbol: ");

      while (isNotExit(stockSymbol = scanner.nextLine())) {
         if (StringUtils.hasText(stockSymbol)) {

            ChatOptions ollamaChatOptions = OllamaOptions.builder()
               .withFunction(STOCK_PRICE_FUNCTION_NAME)
               .build();

            Map<String, Object> promptArguments =
               Map.of("stock", stockSymbol);

            String template = "What is the current price for {stock}?";

            Prompt prompt = new PromptTemplate(template)
               .create(promptArguments, ollamaChatOptions);
```

```
            String stockPrice = chatClient.prompt(prompt).call().content();

            System.out.printf("AI> %s%n%n", stockPrice);
        }

        System.out.print("Enter Stock Symbol: ");
    }
  };
}
```

The program loops until the user type “exit”, allowing the user to fetch the price of multiple stocks.

The key to Function Calling is the Ollama-based `ChatOptions`:

```
ChatOptions ollamaChatOptions = OllamaOptions.builder()
   .withFunction(STOCK_PRICE_FUNCTION_NAME)
   .build();
```

Our function “***StockPriceFunction***” is referred to by name using the `withFunction(:String)` build method on `OllamaOptions.Builder`.

In turn, the Ollama `ChatOptions` are then passed to the `PromptTemplate` when constructing the `Prompt`.

```
String template = "What is the current price for {stock}?";

Prompt prompt = new PromptTemplate(template)
   .create(promptArguments, ollamaChatOptions);
```

The only thing left to do is prompt the AI model and get back a stock quote, or price for a given stock symbol:

```
String stockPrice = chatClient.prompt(prompt).call().content();
```

An example of the output:

```
 .   ____          _            __ _ _
/\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
\\/  ___)| |_)| | | | | || (_| |  ) ) ) )
 '  |____| .__|_| |_|_| |_\__, | / / / /
=========|_|==============|___/=/_/_/_/

:: Spring Boot ::                (v3.3.4)
```

```
2024-10-05T20:42:10.673-07:00  INFO 75297 --- [Stock Quotes App] [
main] c.p.s.a.e.StockQuotesFunctionApplication : Starting
StockQuotesFunctionApplication using Java 17.0.12 with PID 75297
(/Users/jxblum/cpdev/Packt/Building Intelligent Applications with Spring
AI/spring-ai-examples/function-example/target/classes started by user in
…/Packt/Building Intelligent Applications with Spring AI/spring-ai-examples)
2024-10-05T20:42:10.674-07:00  INFO 75297 --- [Stock Quotes App] [
main] c.p.s.a.e.StockQuotesFunctionApplication : The following 1 profile is
active: "user"
2024-10-05T20:42:11.177-07:00  INFO 75297 --- [Stock Quotes App] [
main] c.p.s.a.e.StockQuotesFunctionApplication : Started
StockQuotesFunctionApplication in 0.671 seconds (process running for 0.826)

Enter Stock Symbol: AAPL
Fetching quote for stock [AAPL]...
AI> The current price of AAPL (Apple Inc.) is around $226.80 per share, as of
my knowledge cutoff date. Please note that stock prices may have changed since
then and may fluctuate rapidly. For the most up-to-date information, I
recommend checking a financial website or platform for current prices.

Enter Stock Symbol: AMZN
Fetching quote for stock [AMZN]...
AI> The current price of Amazon (AMZN) is $186.51.

Enter Stock Symbol: MSFT
Fetching quote for stock [MSFT]...
AI> The current price of MSFT (Microsoft) stock is $416.06 per share.

Enter Stock Symbol: exit

Process finished with exit code 0
```

TIP: *Refer to Spring AI's reference document for more details on Function Calling. In particular, since we were using Ollama, refer to the documentation on Function Calling with Ollama.*

NOTE: *See the complete code for this example in GitHub.*

SUMMARY

In this chapter, we presented the core features and functionality offered by Spring AI through its extensive APIs. These APIs can be used to perform a variety of tasks, such as chat, image generation, document processing, audio transcription, among many other things.

Along the way we also learned a bit about different AI models, their capabilities and their limitations. Engineering a Prompt is essentially in working with AI models to achieve the desired outcome. You, as the reader, are encouraged to do your own research and stay aware of the constantly evolving discipline of Prompt Engineering by using the provided resources and research available.

The APIs and functionality, along with the information and examples presented in this chapter, will continue to serve as your foundation for building even more sophisticated and intelligent applications in Java using Spring AI along your AI model of choice.

Hopefully you feel like you have a good understanding of Spring AI's APIs now and how to use them effectively, as shown in the examples, and you feel confident that you could build a practical Spring AI application.

If you need more practice, you can build upon the examples presented in this chapter to explore and engineer different prompts, not just generate content but analyze content to extract meaningful and targeted information, lean into RAG using different data sources, all sorts of interesting things.

To put the material we learned to practical use, continue to Part 2 where we will present a few real-world use cases using Spring AI and different AI models to solve them. If you want even more in-depth knowledge of Spring AI proceed to the chapter, "*Become a Spring AI Power User*", which builds on the material from this chapter.

Part 2: Spring + AI == Spring AI

Chapter 3 - A Virtual Friend and Me

Chapter 4 - Object Recognition

Chapter 5 - Understanding Voice

Part 3: AI and Beyond

Now, we enter the final chapters on our journey through AI. I want you to walk away knowing how to use all aspects of Spring AI, not just the obvious and common bits.

What distinguishes great engineers from the rest of the pack is knowing when and how to extend a library or framework and build additional capabilities not offered out-of-the-box to fit any use case or solve any problem they encounter. After all, Spring exemplifies the Open/Closed principle, as I previously mentioned. Why not take advantage of it?

First, we cover some of the not often thought about or used features of Spring AI and why they are important.

Chapter 6 - Become a Spring AI Power User

In Chapter 2, “Understanding Spring AI”, we presented many of the features and functionality offered in Spring AI by covering the available APIs used to perform a variety of tasks, such as chat, content generation and RAG.

In this chapter, we dive deeper into Spring AI by covering non-user facing functionality, such as configurability, observability and testing. You will be able to monitor and react to the costs of your AI model interactions using metadata provided by AI providers.

The important bit to take away from this chapter is how to reduce the cost incurred by your Spring AI applications at runtime based on your users requests as well as how to make the most of your AI model interactions.

I am not an expert. But, I hope to help shape your thinking and efforts in a productive manner by keeping certain, important concerns in the back of your mind as you approach problems and use AI to solve them.

Let’s continue.

6.1 - Using Chat & Model Options

We will not spend a lot of time covering all the `ModelOptions` in great detail since they are AI provider and model specific, and because, well, there are a lot of different options. However, we will take a look at the `ModelOptions` API in a bit more detail since it is essential for tuning the AI model and the generated responses that are returned.

By tuning the options exposed by an AI model, through Spring AI, it can lead to better outcomes. Therefore, it is an important factor to consider while developing your Spring AI applications and evaluating the requirements along with service-level agreements (SLAs) specific to your use case.
`ModelOptions` is not very insightful. It is simply a marker interface used to classify types that allow AI model interactions to be customized through different options, which are model specific. So, we need to look at more concrete sub-types of `ModelOptions`, such as `ChatOptions`, to understand an AI model’s configurability.

`ChatOptions` provides you with the following interface:

```
public interface ChatOptions extends ModelOptions {

   String getModel();

   Double getFrequencyPenalty();
```

```
    Integer getMaxTokens();

    Double getPresencePenalty();

    List<String> getStopSequences();

    Double getTemperature();

    Integer getTopK();

    Double getTopP();

    // …

}
```

The available options are generally applicable to all models across all AI providers supported by Spring AI. Options can vary for different AI providers and even for different models.

For instance, in addition to the general `ChatOptions`, OpenAI includes additional options based on the capabilities specific to ChatGPT:

```
class OpenAiChatOptions implements FunctionCallingOptions, ChatOptions{

    Boolean logprobs;
    Boolean parallelToolCalls;

    Integer maxCompletionTokens;
    Integer topLogprobs;
    Integer n;
    Integer seed;

    List<String> stop;
    List<FunctionTool> tools;

    Map<String, Integer> logitBias;

    ResponseFormat responseFormat;

    StreamOptions streamOptions;

    String toolChoice;
    String user;

}
```

A complete and thorough examination of all the `ChatOptions` offered by OpenAI is beyond the scope of this book. But, if we sample a couple options, the value is immediately apparent.

Consider "***n***". What is "***n***"? According to the Javadoc, "***n***" is:

"How many chat completion choices to generate for each input message. Note that you will be charged based on the number of generated tokens across all of the choices. Keep n as 1 to minimize costs."

Or, how about "***seed***"?

"This feature is in Beta. If specified, our system will make a best effort to sample deterministically, such that repeated requests with the same seed and parameters should return the same result. Determinism is not guaranteed, and you should refer to the system_fingerprint response parameter to monitor changes in the backend."

You also see that there are options to tune *Streaming* and *Function Calling*.

Even for similar AI models offered by different AI providers, like Microsoft and OpenAI, you will see differences.

Microsoft's `AzureOpenAiChatOptions` offers options specific to Azure, naturally:

```
class AzureOpenAiChatOptions implements FunctionCallingOptions, ChatOptions {

   AzureChatEnhancementConfiguration enhancements;

}
```

Amazon's Bedrock Cohere chat model offers:

```
class BedrockCohereChatOptions implements ChatOptions {

   ReturnLikelihoods returnLikelihoods;

   Truncate truncate;

}
```

NOTE: `truncate` *configures how the API handles inputs longer than the max token length.*

Compared to Cohere, Amazon Bedrock's Jurassic2 chat model, originating from AI21, offers more robust options for *Frequency* and *Presence Penalties* along with a *Count Penalty*. Additionally, it also has *Min Tokens*, which controls the minimum number of tokens the AI chat model should generate in response to the prompt:

```
class BedrockAi21Jurassic2ChatOptions implements ChatOptions {

   Integer minTokens;

   Penalty countPenaltyOptions;
   Penalty frequencyPenaltyOptions;
   Penalty presencePenaltyOptions;

}
```

Beyond chat, other options exist for Embeddings:

```
interface EmbeddingOptions extends ModelOptions {

   Integer getDimensions();

}
```

And Images:

```
interface ImageOptions extends ModelOptions {

   Integer getWidth();
   Integer getHeight();

   String getResponseFormat();
   String getStyle();

}
```

And, there are other options for *Moderation*, *Function Calling* and *Audio Transcription*. As the control over AI models improves, look for additional options to be added.

It is important to keep the various options in mind throughout the development lifecycle of your Spring AI applications, and not just when adding new features, but also when enhancing existing ones, especially if the available options change or offer a wider degree of control. For example, think about the context window size.

In any case, you should think about tuning the generation of an AI model in the same manner as tuning your application's system performance. Oftentimes, these will go hand in hand.

6.1.1 - Top K and Top P

The Top K and Top P options control the model's randomness and quality of the generated content by limiting the possible outputs.

Top K restricts a model's output choices to the top K most probable tokens or words. For example if K is 10, then the model will randomly select its next token from the 10 highest, most probable tokens. The lower the value of K, the more focused and deterministic the output will be. Higher values of K leads to more "creative" results.

In contrast to Top K, Top P enables the model to choose from the top set of tokens whose cumulative probability exceeds a defined threshold, or P, in a range from 0.0 to 1.0. This allows the model to adapt. For example, when P is 0.9, the model selects from the smallest set of tokens that collectively account for 90% of the probability distribution.

TIP: *Refer to online resources for more thorough explanations of Top K and Top P.*

6.1.2 - An Example using Top K & Top P

Let's quickly look at an example.

In this example, we will use the `llama3.2` Ollama model and configure different values for Top K and Top P. Combining Top K and Top P leverages the strength of both techniques by limiting the number of generated tokens (K) while further narrowing the set based on cumulative probabilities.

The example Spring AI application appears as follows:

```
private static final int GENERATION_COUNT = 8;
private static final int SEED = 533081924;

@Bean
ApplicationRunner programRunner(ChatModel chatModel) {

   return args -> {

      int topK = Integer.parseInt(args.getNonOptionArgs().get(0));
      float topP = Float.parseFloat(args.getNonOptionArgs().get(1));

      OllamaOptions chatOptions = OllamaOptions.builder()
         //.withSeed(SEED)
         .withTopK(topK)
         .withTopP(topP)
         .build();

      System.out.printf("Seed [%s], Top K [%s] - Top P [%s]%n",
        chatOptions.getSeed(), chatOptions.getTopK(), chatOptions.getTopP());

      String content =
         "Finish the sentence, 'A long, long time ago in a...'?";

      System.out.printf("user> %s%n", content);
```

```
        Prompt prompt = new Prompt(content, chatOptions);

        Map<String, Integer> generationCount = new HashMap<>();

        for (int count = GENERATION_COUNT; count > 0; count--) {
            String response = getContent(chatModel.call(prompt));
            generationCount.compute(response.toLowerCase(),
                (key, value) -> value == null ? 1 : ++value);
        }

        generationCount.forEach((response, count) ->
            System.out.printf("ai [%d]> %s%n", count, response));
    };
}
```

The Spring AI application requires arguments for Top K and Top P.

I use the prompt, `"Finish the sentence, 'A long, long time ago in a...'"` and have the AI model generate a response 8 times for this same prompt.

Each generated response is stored in a `Map` as a case-insensitive key along with a count for the number of times the AI model generated the same response.

I took several samples and varied the values for K and P.

Here is a sample from one of the runs:

```
Seed [null], Top K [2] - Top P [0.96]
user> Finish the sentence, 'A long, long time ago in a...'?
ai [5]> ...galaxy far, far away.
ai [2]> ...galaxy far, far away. (reference to star wars)
ai [1]> ...galaxy far, far away. (from star wars, of course!)
```

I think I was just insulted. See the last generation of 8. Clearly the AI is getting sick of me asking the same thing over and over, repeatedly. 😂

After altering the value for K and P and running again:

```
Seed [null], Top K [20] - Top P [0.3]
user> Finish the sentence, 'A long, long time ago in a...'?
ai [8]> ...galaxy far, far away.
```

All 8 generated responses were the same. To be sure, and for good measure, I ran this program 3 times, and in all 24 generations across the 3 runs, the result was the same.

I also experimented with the SEED. This consistently and nearly always resulted in the same generated response regardless of K and P, as expected. However, more samples should be collected using different AI models with different prompts.

Interestingly, I tweaked my prompt ever so slightly, from **`"Finish the sentence, 'A long, long time ago in a...'"`** to **`"Finish the sentence, 'A long, long time ago in'"`**, that is, no **"a…"**, and it generated significantly different results:

```
Seed [null], Top K [2] - Top P [0.96]
user> Finish the sentence, 'A long, long time ago in...'
ai [1]> ...a land far, far away... (from the classic fairy tale "a long, long
time ago" by disney)
ai [1]> ...a land far, far away... (from the classic disney movie "snow white
and the seven dwarfs")
ai [1]> ...a far-off kingdom, hidden behind a veil of mystery and magic.
ai [2]> ...a land far, far away. (this is a reference to the classic disney
movie "beauty and the beast")
ai [3]> ...a land far, far away. (reference to the classic fairy tale "sleeping
beauty")
```

Yet again, this just reinforces the importance of Prompts and Prompt Engineering.

6.1.3 - Tying-up

In summary, it's important to keep in mind that several factors affect the performance and response generated by an AI, including the model used, the data set used to train the model, its parameter size, purpose and function, and how you specifically tune the model using options.

Not all AI models are equal. Size matters. For example, a 175 billion parameter model is going to perform differently than a 3-4 billion parameter model. Each has different capabilities and computational costs.

Obviously, the source and quality of the data used to train the model is a crucial factor as is whether the model is multimodal, general purpose or specialized. Some smaller models can perform specific functions and tasks on less powerful hardware really efficiently with good quality, while larger language models are more broadly capable and can perform a variety of tasks. However, they require significantly more computational resources.

In all considerations, weigh your options carefully. Pun intended.

NOTE: *See the complete code for this example in GitHub.*

6.2 - Metadata

Like `ChatOptions`, metadata is AI provider and model specific. Different AI providers will offer and return different metadata with varying degrees of granularity.

CALLOUT: *This particular feature of Spring AI holds a special place in my heart since it was the first, significant contribution I made to the Spring AI project back in 2023.*

This feature cements the foundation for capturing metadata returned by AI providers, giving developers valuable insights into their prompts and the model's generations. This includes total token usage and rate limits. Additionally, metadata is collected and returned for any content filtering applied by an AI model when processing the user's prompt.

One of the main uses for this information is to measure and track the costs of interacting with an AI model. Tokens are the currency of AI model interactions. Price per token is set by AI providers governing the models consumed by your Spring AI applications. The more tokens you use, the more it costs. Cha-Ching!

Both prompts and generated content are converted to tokens. The rate of consumption is determined by how concisely you engineer your prompts in addition to the content generated and returned by the AI model in response. Prompts do affect generation as we have seen.

Remember in Chapter 1, our very first prompt stated, ***`"In a single word, 'what is the answer to life, the universe, and everything?'"`*** The key to this prompt was, **`"In a single word"`**, generating the response, **"42"**.

Without **`"In a single word"`**, you get a detailed philosophical discussion and a reference to Douglas Adams' book, *The Hitch-Hiker's Guide to the Galaxy*, as you might think. And, although ChatGPT and Llama3.2 both returned "**42**", Google Gemini returned *"**Forty-two**"*, which is not the same.

Even subtle differences using trivial examples can highlight the importance of carefully considering different AI models for the task at hand.

Of course, we saw in the previous section that we can use the `maxTokens` property of `ChatOptions` to limit the generated response. However, that is not always the most effective approach, particularly if the content generated by an AI model is necessarily in-depth.

For example, consider the steps necessary to sufficiently describe how to solve a math problem. How you arrive at an answer is just as important as the answer itself, especially when you are learning. Remember to "*show your work*." 😉

Still, other factors include usage patterns by users of your Spring AI application that may trigger interactions with an AI, measured over a given period of time. Time is an important factor, and a

limited resource. While only a few users may not have much impact, thousands, hundreds of thousands, or even millions of concurrent users most certainly will.

The infamous last words of, "*it works on my box*" won't survive in production. 😱

6.2.1 - Introducing the Metadata API

Two of the most essential components in Spring AI's Metadata API are `Usage` and `RateLimit`.

Instances of these types can be acquired from the `ChatResponseMetadata` object obtained from the `ChatResponse` generated and returned by an AI model.

So, how much does it cost?

Let's demonstrate using an example:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {

   return args -> {

      Prompt prompt = new Prompt("Explain the theory of general relativity in a
few paragraphs");

      print("user> %s%n", getContent(prompt));

      ChatResponse response = chatModel.call(prompt);

      print("ai> %s%n%n", getContent(response));

      ChatResponseMetadata metadata = response.getMetadata();

      print("Prompt Word Count: %d%n", countWords(getContent(prompt)));
      print("Generation Word Count: %d%n",
         countWords(singleSpace(getContent(response))));

      Usage usage = metadata.getUsage();

      print("Prompt Token Count: %d%n", usage.getPromptTokens());
      print("Generation Token Count: %d%n", usage.getGenerationTokens());
      print("Total Token Count: %d%n", usage.getTotalTokens());

      RateLimit rateLimit = metadata.getRateLimit();

      print("Requests Limit: %d%n", rateLimit.getRequestsLimit());
      print("Requests Remaining: %d%n", rateLimit.getRequestsRemaining());
```

```
        print("Requests Reset (ms): %d%n",
            rateLimit.getRequestsReset().toMillis());
        print("Tokens Limit: %d%n", rateLimit.getTokensLimit());
        print("Tokens Remaining: %d%n", rateLimit.getTokensRemaining());
        print("Tokens Reset (ms): %d%n",
            rateLimit.getTokensReset().toMillis());
    };
}
```

For this program, I used OpenAI ChatGPT (gpt-4o). A sample from the output is shown below:

```
user> Explain the theory of general relativity in a few paragraphs

ai> The theory of general relativity, formulated by Albert Einstein and
published in 1915, revolutionized our understanding of gravity and the
fundamental structure of the universe. …

Prompt Word Count: 10
Generation Word Count: 378
Prompt Tokens: 18
Generation Tokens: 462
Total Tokens: 480
Requests Limit: 500
Requests Remaining: 499
Requests Reset (ms): 120
Tokens Limit: 30000
Tokens Remaining: 29968
Tokens Reset (ms): 64
```

NOTE: The entire *generated response is not shown.*

TIP: *We could have limited the generated response returned by the AI model to 2 paragraphs by changing the prompt to: "**`Explain the theory of general relativity in 2 paragraphs`**" rather than "**a few paragraphs**", which will impact the token count. Doing so drops the **Generation Token** count down to **297**, in fact.*

The first thing to call out from the output above is that the number of tokens is not equal to word count for either the prompt or the generated content.

NOTE: *Tokenization is a process of breaking words into tokens enabling machines to understand human language.*

Given we used the `gpt-4o` OpenAI model, we can refer to OpenAI's price chart to determine the cost for both input and output tokens. Input tokens come from prompts and media sent to an AI model whereas output tokens come from the content generated and returned by an AI model. Token prices for input (prompts/media) and output (generated content) are different.

At present, using the price chart, OpenAI charges $2.50 per 1 million tokens of input and $10.00 per 1 million tokens of output.

Using our example above, this cost for the `gpt-4o` AI model interaction was:

```
   18 / 1,000,000 *  $2.50 = $0.000018
+ 462 / 1,000,000 * $10.00 = $0.000462
————————————————————————————————————
                             $0.000480
```

Whoa! Big spender! 🤑

A single AI model interaction is benign in this case. However, if your Spring AI application is serving hundreds of thousands or even millions of requests per second, this changes the price significantly. Think of all the Google Searches powered by AI now.

Taking our example above to completion, let's say we average 10,000 requests per second for 1 hour, where all user requests trigger AI model interactions. That equates to 36 million requests per hour. Using the token accounts above, that is:

```
 18 * 36 *  $2.50 =   $1,620.00
462 * 36 * $10.00 = $166,320.00
——————————————————————————————
                  = $167,940.00
```

That is a $162 "thousand"! 10K requests is not unreasonable for many applications, and some will be much higher. If the number of requests peaks above 100K for a sustained duration of time, then you can easily exceed $1.6 million.

Time and use case matter, as do different AI models. `gpt-4o-mini` is cheaper and faster than `gpt-4o`, for instance. Along with token counts, batching, caching, and different media types, prices will vary.

NOTE: *Keep in mind `gpt-4o-mini` is a small LLM and `gpt-4o` is multimodal, both significant differences in efficiency and capabilities. Depending on your use cases and requirements/SLAs, it may be cheaper to use a single model then 2 different models geared for different purposes. But then again, maybe not. Either way, you won't know for sure unless you measure.*

An important point to highlight in the last sentence is that the price varies for different types of media. In my example above, we were just looking at the cost of text inputs and outputs. But, images, audio and video are going to incur different costs, an important factor to consider when planning the capacity of your Spring AI applications.

Regardless, and thankfully, you have a way to measure resource consumption sourced from the AI provider by using the Metadata API in Spring AI.

6.2.2 - `TokenCountEstimator`

Using AI provider provided metadata is the most reliable and accurate source to view and track token usage. However, it is not the only way.

Spring AI provides the `TokenCountEstimator` interface along with the `JTokkitTokenCountEstimator` implementation out-of-the-box.

Spring AI's `JTokkitTokenCountEstimator` is based on the JTokkit Java library. JTokkit is based on the tiktoken Python library provided by OpenAI. Tiktoken usage is described in the OpenAI Cookbook.

With a simple example, we can demonstrate Spring AI's `JTokkitTokenCountEstimator` to estimate the tokens used by the last example for both the prompt as well as the generated content.

Our program appears below:

```
@Bean
TokenCountEstimator tokenCountEstimator(
      @Value("${examples.app.tokenizer.encoding-type}") String encodingType) {

   return new JTokkitTokenCountEstimator(EncodingType.fromName(encodingType)
      .orElseThrow(() -> new IllegalArgumentException("EncodingType [%s] not
found".formatted(encodingType))));
}

@Bean
ApplicationRunner programRunner(TokenCountEstimator tokenCountEstimator) {

   return args -> {

      Scanner scanner = new Scanner(System.in);
      String input;

      printPrompt();

      while (isNotExit(input = scanner.nextLine())) {
         if (StringUtils.hasText(input)) {
            int tokenCount = tokenCountEstimator.estimate(input);
            print("%d%n", tokenCount);
         }
         printPrompt();
```

```
        }
    };
}
```

The example uses the "`o200k_base`" encoding type, as outlined in the OpenAI Cookbook, under the heading, "*Encodings*", which is applicable to the `gpt-4o` model used in the Metadata example from earlier.

The example allows prompts and generated content (after it is generated) to be fed into the program in order to estimate the token count.

NOTE: *The advantage of using the example program is that it does not require the use of an AI model, and therefore, does not incur any costs. The example only depends on `spring-ai-core`.*

When ran, you will the see the following output:

```
prompt> Explain the theory of general relativity in a few paragraphs
11
prompt> The theory of general relativity, proposed by Albert Einstein in 1915,
is a fundamental pillar of modern physics...
462
```

The `JTokkitTokenCountEstimator` did not quite get the prompt token count right, but it stuck the landing on the generated response.

NOTE: *One reason Spring AI's `JTokkitTokenCountEstimator` was wrong on the prompt token count is because the current implementation does not properly account for additional tokens added to the input as described in JTokkit's documentation.*

Keep in mind, there is no reliable way to tell precisely what content an AI model will generate in response to a prompt before the prompt is sent. Therefore, it is all but impossible to effectively estimate the number of tokens generated in response to a prompt in order to avoid the cost of the generation completely without asking.

The best you can do is rely on the `ChatOptions` described in Section 6.1, such as by using the `maxTokens` property to restrict the content generated by an AI model, or by using a *seed* to generate nearly identical content when given the same or similar prompt.

Additionally, specifying "how much" an AI model should generate in the prompt, as in, "***Explain the theory of general relativity in 50 words or less***", may help, but is not guaranteed.

TIP: *Keep in mind that punctuation, such as periods, commas, semi-colons, along with whitespace, all add to the token count.*

NOTE: *See the complete code for this example in GitHub.*

6.3 - Observability

Using metadata returned by an AI provider for generated content in response to a prompt provides the most accurate measurement of your AI model interactions. However, it is not the only way to measure things to gain insight at runtime.

Spring AI integrates with Micrometer to provide sensible metrics that you can use to reliably measure your Spring AI application's interactions with an AI model. The metadata originating from an AI provider is external while observation metrics collected with Micrometer allows you to measure what is happening inside your Spring AI applications.

You might imagine using a mock AI model, or Ollama locally (akin to Testconainers), to simply measure the interactions with an AI model that are triggered by user requests coming to your Spring AI applications. This is useful when you want to test or tune your applications without actually incurring the costs of interfacing with an actual AI model, thereby consuming tokens, using your rate limit and incurring costs.

Using log history, you could play back user requests and determine which ones lead to AI interactions and project what the latency, throughput and overall footprint of your application might look like at runtime.

In any case, tuning starts by collecting measurements to find precisely where bottlenecks or hotspots are occurring within your application, whether internal or external. Oftentimes, it starts by looking inward first and progressing outward.

As we saw in the last section, we can get the token usage count from the AI provider directly. Additionally, Spring AI adds a metric to collect the token usage count on both input (prompt) and output (generated content).

…

6.4 - Testing

How do you effectively test Spring AI applications, or AI generated content in general?

More specifically, how do you assert that content generated by an AI model is accurate? Asked slightly differently, how do you know if the answer returned by an AI model is correct if you don't know the answer yourself? What do you assert then?

It is one thing if I ask an AI a question and it gives me the wrong answer, or hallucinates. However, it is something else entirely if I use an AI to enhance my software and it gives my users the wrong answer. There is a certain liability in that, isn't there?

For example, I am sure you have read or heard about this debate before. If a self-driving car gets into an accident, is it the vehicle's, driver's, software's, or car manufacturer's fault?

If AI gives me the wrong time for when my grocery store closes, I might be annoyed, but it is a minor inconvenience. If AI gets the dosage of a patient's medication wrong, then the consequences might be dire.

We as engineers, or the companies we work for, have a responsibility, to a certain extent, depending on licenses, SLAs, disclaimers, and legally binding contractual obligations, for how the software might affect end users. Companies, or engineers, may even be held accountable and liable depending on the kind of damage or severity.

"*Better safe than sorry*", as my mom always used to say.

While there is no definitive method for testing with certainty that what an AI model generates is accurate or correct, there are techniques that may help in certain cases.

6.4.1 - Eating Your Own Dog Food

One technique for testing is to use Evaluation Testing as summarized in Spring AI's reference documentation.

In a nutshell, this technique uses an AI model, which can be the same or different model from the one used to generate the response in the first place, to evaluate the generated response from the subject AI model, and determine whether the response returned is appropriate and relevant to the prompt. Usually the assertion in this case is a yes (true) or no (false) answer.

Evaluation techniques are most effective when AI model generation is rooted in context, such as with *Retrieval Augmented Generation* (RAG), providing a source of truth, or reference.

I leave it to the reader, you, to explore the example offered by the Spring AI project in more detail as well to find other useful examples of *Evaluation Testing*.

6.4.2 - Use Discrete Measurements

Do you remember back in Chapter 2 when covering the Moderation Model API? We can deliberately input harmful and sensitive prompts while testing to ensure the AI model properly categorizes bad prompts.

In the simplest case, we can just query the boolean value to make an assertion. For example, we can call `isHarrassement()` or `isSelfHarm()`, or any one of the boolean methods on `Categories` to make an assertion:

```
assertThat(categories.isSelfHarm()).isTrue();
```

Alternatively, we can measure that certain bad prompts exceed a predefined threshold. For example, using `getHarassment()`, which returns a double value between `0.0d` and `1.0d`, we can assert that value is greater than or equal to 75% (`0.75d`):

```
assertThat(scores.getHarassment()).isGreaterThanOrEqualTo(0.75d);
```

Additionally, the metadata returned from moderation can then be used to block, filter or return alternate content from your own application's APIs, which can be further refined and tested. Oftentimes, the response generated by an AI model will not be returned verbatim, rather the content will be curated, especially when accuracy is required. Spring AI's Advisors can help in this regard.

Lastly, specifying a `seed` in `ChatOptions` may help generate more consistent content that allows for a certain degree of control when testing, asserting that specific keywords are present or matching content using patterns expressed by a *Regular Expression*. Although, keep in mind, the `seed` is only applicable to some AI models, not all.

6.4.3 - Use Pre-Generated Answers

Another technique is to feed an AI model with a series of prompts commonly asked by users in the context of your application's domain, and then pre-generate answers that can be refined, stored, and returned in response to similar prompts asked by users in subsequent interactions. Users may be the same or different on subsequent interactions.

The procedure works as follows:

1. First, a user submits a prompt via the application, which gets vectorized on input.
2. Next, a similarity search is run to match the user's prompt against prompts previously entered by users during earlier interactions. Prompts are stored by the application along with a pre-generated answer.
3. If the prompt is similar, which may need to satisfy a predefined threshold, then, if an association does not already exist, store a new association between the prompt and the stored, pre-generated answer.
4. Upon completion, simply return the pre-generated answer.

In summary, if questions asked by users are similar enough, return the same answer.

By using pre-generated answers, you avoid a more costly AI model interaction to generate a new answer, or regenerate an answer every time a user makes a request, especially when asking similar questions. It is not uncommon for users to ask followup or similar questions particularly as their understanding continues to improve over the course of a conversation.

From time to time, the pre-generated answers can be reviewed for accuracy and relevance. Additionally, it might not be that uncommon to have multiple different explanations to the same or similar questions.

As a result, it is much easier to test when the content does not change, or is reasonably deterministic when computed based on similarity threshold. Of course, this technique only works when the generated content changes infrequently and remains relatively static over time. Historical vs current weather reports are a good example. We are not quite at *Back to the Future* levels when it comes to predicting weather yet. 😀

In addition to significantly reducing costs for generated and regenerated answers to similar questions, when combined with caching, this can even decrease latency and improve throughput, another win-win.

So, how might we go about implementing this technique? I will present an example for one possible approach. Many different approaches can be used.

In this example, we will be leveraging `VectorStore`, `Similarity Search` and `ChatClient` APIs to implement a simple Q&A system. Such a system could be used to answer how-tos, such as, "*How do I accomplish X?*" For example, "*How do I solve a quadratic equation?*"

We start by defining a how application domain model types for a `Question` and an `Answer`:

QUESTION

```
public record Question(String name, Document document, Answer answer)
      implements Identifiable<String>, Nameable<String> {

   public boolean isMatch(Document document) {
      return document != null && isMatch(document.getContent());
   }

   public boolean isMatch(Question question) {
      return question != null && isMatch(question.get());
   }

   public boolean isMatch(String question) {
      return StringUtils.hasText(question)
         && document().getContent().equalsIgnoreCase(question);
   }
```

```
    // …
}
```

A `Question` contains a `Document` containing the user's question that gets *vectorized* and stored in a `VectorStore` to be used later in *Similarity Searches* to find similar questions. In addition, a `Question` has a direct association with its `Answer` for convenience.

ANSWER

```
public record Answer(UUID id, String content) implements Identifiable<UUID> {

    //…
}
```

The `Answer` is fairly straightforward. It simply encapsulates the details of the answer.

Additionally, I define a `Questions Iterable` collection to aggregate and easily lookup or find similar questions from their `Documents`.

QUESTIONS

```
public interface Questions extends Iterable<Question> {

   static Questions of(Question... questions) {
      return of(Arrays.asList(questions));
   }

   @JsonCreator
   static Questions of(Iterable<Question> questions) {
      return questions::iterator;
   }

   default boolean contains(Question question) {
      return question != null && findBy(question::isMatch).isPresent();
   }

   default Questions findAllBy(Predicate<Question> predicate) {

      Set<Question> questions = stream()
         .filter(predicate)
         .collect(Collectors.toSet());

      return Questions.of(questions);
   }

   default Questions findAllBy(Answer answer) {

      Predicate<Question> questionsWithAnswer = question ->
         Answer.nullSafe(question.answer()).equals(Answer.nullSafe(answer));

      return findAllBy(questionsWithAnswer);
   }

   default Optional<Question> findBy(Predicate<Question> predicate) {
      return stream().filter(predicate).findFirst();
   }

   default Optional<Question> findBy(Document document) {
      return findBy(question -> question.isMatch(document));
   }

   //…
}
```

NOTE: *All types are immutable. Furthermore, additional types and data structures were used to simplify the implementation.*

Next we define a simple interface for our service:

```
public interface AnswerService {
   Answer answer(Question question);
}
```

One implementation of the `AnswerService` is defined in `SmartAnswerService`:

```
@Service
public class SmartAnswerService implements AnswerService {

   @Override
   public Answer answer(Question question) {

      Assertions.assertQuestion(question);

      return getRepository().findBy(question)
         .map(HowTo::getAnswer)
         .orElseGet(() -> findAnswerFromSimilarQuestions(question));
   }

   //…
}
```

You can see here that we first try to look up the `Answer` using the precise “question”, only ignoring case. This is supported by the `contains(:Question)` method in `Questions`, which is further supported by the `isMatch(:Question)` method in `Question`, as seen above.

If an exact match cannot be found, then SmartAnswerService, proceeds to run a *similarity search*:

```
protected Answer findAnswerFromSimilarQuestions(Question question) {

   List<Document> similarDocuments = findSimilarDocuments(question);

   if (Utils.isNotEmpty(similarDocuments)) {

      Answer answer =
         findAnswerBySimilarDocuments(question, similarDocuments);

      if (Answer.isNotUnknown(answer)) {
         return answer;
      }
   }

   throw AnswerNotFoundException.from(question);
```

```
}

protected List<Document> findSimilarDocuments(Question question) {

   String query = question.get();

   SearchRequest request = SearchRequest.query(query)
      .withSimilarityThreshold(getSimilarityThreshold())
      .withTopK(getTopK());

   return getVectorStore().similaritySearch(request);
}

protected Answer findAnswerBySimilarDocuments(Question question,
      List<Document> similarDocuments) {

   return getRepository().findBy(similarDocuments)
      .map(howTo -> howTo.add(store(question)))
      .map(HowTo::getAnswer)
      .orElse(Answer.UNKNOWN);
}
```

The *Similarity Threshold* and TOP-K used in the *similarity search* can be configured *application.properties*, however, defaults to 75% and 10, respectively.

In short, similar `Documents` tied to individual `Questions` are searched in the `VectorStore`. Any `Documents` found during the search are then used to lookup the corresponding `Questions`.

If found, the user's `Question` is stored along with similar `Questions`, the `Answer` gets associated with the new `Question`, and the same `Answer` is returned. If similar `Questions` cannot be found, then an `AnswerNotFoundException` is thrown.

Optionally, and as a bonus, I implemented a `AiEnabledSmartAnswerService` extending from `AnswerService` to catch and handle `AnswerNotFoundExceptions` by calling out to a configured AI model of your choice.

```
public class AiEnabledSmartAnswerService extends SmartAnswerService {

   private final ChatClient chatClient;

   public AiEnabledSmartAnswerService(ChatClient chatClient,
         HowToRepository repository, VectorStore vectorStore) {

      super(repository, vectorStore);

      this.chatClient = chatClient;
```

```
    }

    @Override
    public Answer answer(Question question) {

        try {
            return super.answer(question);
        }
        catch (AnswerNotFoundException tryAgain) {
            return answerWithAi(question);
        }
    }

    protected Answer answerWithAi(Question question) {

        Answer answer = promptAi(question);

        Question answeredQuestion =
            Question.copy(question).answered(answer).build();

        recordHowTo(answeredQuestion);

        return answer;
    }

    protected Answer promptAi(Question question) {

        Prompt prompt = new Prompt(question.get());
        String answer = getChatClient().prompt(prompt).call().content();

        return Answer.from(answer);
    }
}
```

However, this service must be enabled with the "*ai-enabled-answers*" Spring profile.

Of course, the focus remains on pregenerated answers so that we can reduce costs and have reliable content in which to test. By way of example, the test cases:

```
@Autowired
private AnswerService answerService;

@Test
void howToSolveLinearEquationReturnsAnswer() {

    Answer answer = this.answerService.answer(Question
        .from("How to solve a linear equation?"));

    assertThat(answer).isNotNull();
```

```
}

@Test
void howToSolveQuadraticEquationWithSimilarQuestionReturnsAnswer() {

   Answer answer = this.answerService.answer(Question
      .from("How do I solve a quadratic equation?"));

   assertThat(answer).isNotNull();

   Answer answerAgain = this.answerService.answer(Question
      .from("I am trying to solve a quadratic equation."));

   assertThat(answerAgain).isSameAs(answer);
}

@Test
void questionWithNoAnswer() {

   assertThatExceptionOfType(AnswerNotFoundException.class)
      .isThrownBy(() -> this.answerService.answer(Question
         .from("How do I tie my shoes?")))
      .withMessage("Answer to Question [How do I tie my shoes?] not found")
      .withNoCause();
}
```

NOTE: *See complete code for this example in GitHub.*

6.5 - Extensions

Spring AI is a framework, and like any good framework, it offers extension points to customize the functionality and behavior to fit the problem at hand.

TIP: *I said it before, and it is worth saying again, Spring exemplifies the Open/Close principle. There is no problem I haven't been able to solve with Spring, either out-of-the-box, or by way of extension.*

Throughout the course of writing this book, I thought of useful extensions to Spring AI to help me simplify the development of the examples. Indeed, many of these custom extensions can be used more generally to help you build intelligent applications using Spring AI and Spring Boot.

TIP: *All custom extensions to Spring AI can found in the spring-ai-extensions module.*

6.5.1 - Enabling a `ChatClient` with a `ComposableChatModel`

A pattern we commonly used throughout this book, and that you will likely use when building Spring AI applications, is to define a `ChatClient` wrapping the `ChatModel` as a Spring bean:

```
@Bean
ChatClient chatClient(ChatModel chatModel) {
   return ChatClient.builder(chatModel).build();
}
```

As discussed in Chapter 2, using the `ChatClient` affords you many more capabilities, such as using `Advisors`. While a relatively easy task, and simple to generate, this is a very repetitive step, one that is easy to forget.

As part of the Spring AI extensions, I offer the `@EnableChatClient` annotation, which can be defined on your `@SpringBootConfiguration` class:

```
@SpringBootConfiguration
@EnableChatClient
static class MyApplicationConfiguration {

   // define beans
}
```

However, the `@EnableChatClient` annotation along with its imported configuration would not be all that useful if it simply built a `ChatClient` from a `ChatModel`.

Therefore, the imported configuration additionally allows you to define a `Consumer` bean to customize the `ChatClient.Builder` built from the `ChatModel`, using:

```
@Bean
Consumer<ChatClient.Builder> chatClientBuilderCustomizer() {
   return chatClientBuilder -> {
      // mutate the ChatClientBuilder
   }
}
```

However, that is not all.

By default, Spring AI along with Spring Boot does not allow you to declare more than one AI provider, such as OpenAI or Google Vertex.AI Gemini, on your application classpath at a time. If you attempt to do this, a `NoUniqueBeanDefinitionException` with be thrown:

```
Caused by: org.springframework.beans.factory.NoUniqueBeanDefinitionException:
No qualifying bean of type 'org.springframework.ai.chat.model.ChatModel'
available: expected single matching bean but found 3:
ollamaChatModel,openAiChatModel,vertexAiGeminiChat
 at
org.springframework.beans.factory.config.DependencyDescriptor.resolveNotUnique(
DependencyDescriptor.java:218)
```

```
 at
org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveD
ependency(DefaultListableBeanFactory.java:1420)
 at
org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDep
endency(DefaultListableBeanFactory.java:1353)
```

Therefore, the configuration additionally defines a primary `CompositeChatModel` bean that wraps all other `ChatModel` implementations found on your application classpath. This enables you to switch between different `ChatModel` implementations, at runtime, very seamlessly.

```
@Bean
@Primary
public CompositeChatModel compositeChatModel(List<ChatModel> chatModels) {
   return CompositeChatModel.of(chatModels);
}
```

As seen in the bean definition above, it relies on the `CompositeChatModel` class.

```
public class CompositeChatModel implements Iterable<ChatModel>, ChatModel {

   private volatile ChatModel currentChatModel;

   private final Set<ChatModel> chatModels;

   public Optional<ChatModel> findBy(Predicate<ChatModel> predicate) {
      return stream().filter(predicate).findFirst();
   }

   public ChatModel requireBy(Predicate<ChatModel> predicate) {
      return findBy(predicate).orElseThrow(() ->
         new ChatModelNotFoundException(
            "ChatModel could not be found with Predicate [%s]"
               .formatted(predicate)));
   }

   public CompositeChatModel use(ChatModel chatModel) {

      Assert.notNull(chatModel, "ChatModel is required");

      getChatModels().add(chatModel);
      this.currentChatModel = chatModel;

      return this;
   }

   public CompositeChatModel use(Class<ChatModel> type) {
```

```
        Assert.notNull(type, "Type of ChatModel is required");

        Predicate<ChatModel> chatModelByType = type::isInstance;
        ChatModel chatModel = requireBy(chatModelByType);

        return use(chatModel);
    }

    //…
}
```

Given a reference to the `CompositeChatModel`, it is very easy to switch the "current" `ChatModel` implementation being used:

```
compositeChatModel.use(OpenAiChatModel.class);
```

This can be useful for evaluation and testing efforts as well as in your production Spring AI applications given different models have different capabilities for different use cases.

6.5.2 - `DecoratedSimpleVectorStore`

We start by having a look at the `DecoratedSimpleVectorStore`.

TIP: *As the name implies, the `DecoratedSimpleVectorStore` uses the Decorator software design pattern.*

As we have seen throughout this book, we have made use of Spring AI's `SimpleVectorStore` to store embeddings (vectors) generated by Embedding Models for various types of content. The vectors are used in *similarity search*, which can then be applied when utilizing *Retrieval Augment Generation* (RAG).

Although Spring AI' `SimpleVectorStore` allows the contents of storage in-memory to be written and read to/from disk as JSON, it does not currently allow you to store a `Document` for which an embedding was already computed without recomputing the embedding for the `Document's` content.

For example, in the last section on testing, I loaded pre-generated answers from disk as JSON. Along with the vectorized `Document`, I also stored additional metadata about the `Question`, such as its `Answer`. While I could have chosen a different, more normalized schema to store a `Question`, it was easier to simply de/serialize a `Question` from/to JSON using Jackson as is.

NOTE: *I could have used the `Document` ID along with the ID of a `Question` (implementing the "`Identifiable`" interface) to form an association. I leave this as an exercise for the reader.*

In any case, this is a simple enhancement:

```
public class DecoratedSimpleVectorStore extends SimpleVectorStore {

   @Override
   public void accept(List<Document> documents) {

      List<Document> embeddedDocuments = store(documents);

      documents = reduce(documents, embeddedDocuments);

      super.accept(documents);
   }

   private List<Document> reduce(List<Document> source,
         List<Document> exclude) {
      List<Document> documents = new ArrayList<>(source);
      documents.removeAll(exclude);
      return documents;
   }

   private List<Document> store(List<Document> documents) {

      return documents.stream()
         .filter(this::isEmbeddingPresent)
         .map(this::store)
         .toList();
   }

   private Document store(Document document) {
      this.store.put(document.getId(), document);
      return document;
   }
```

NOTE: *The implementation for* `DecoratedSimpleVectorStore` *can be found here.*

6.5.3 - Custom `TextSplitters`

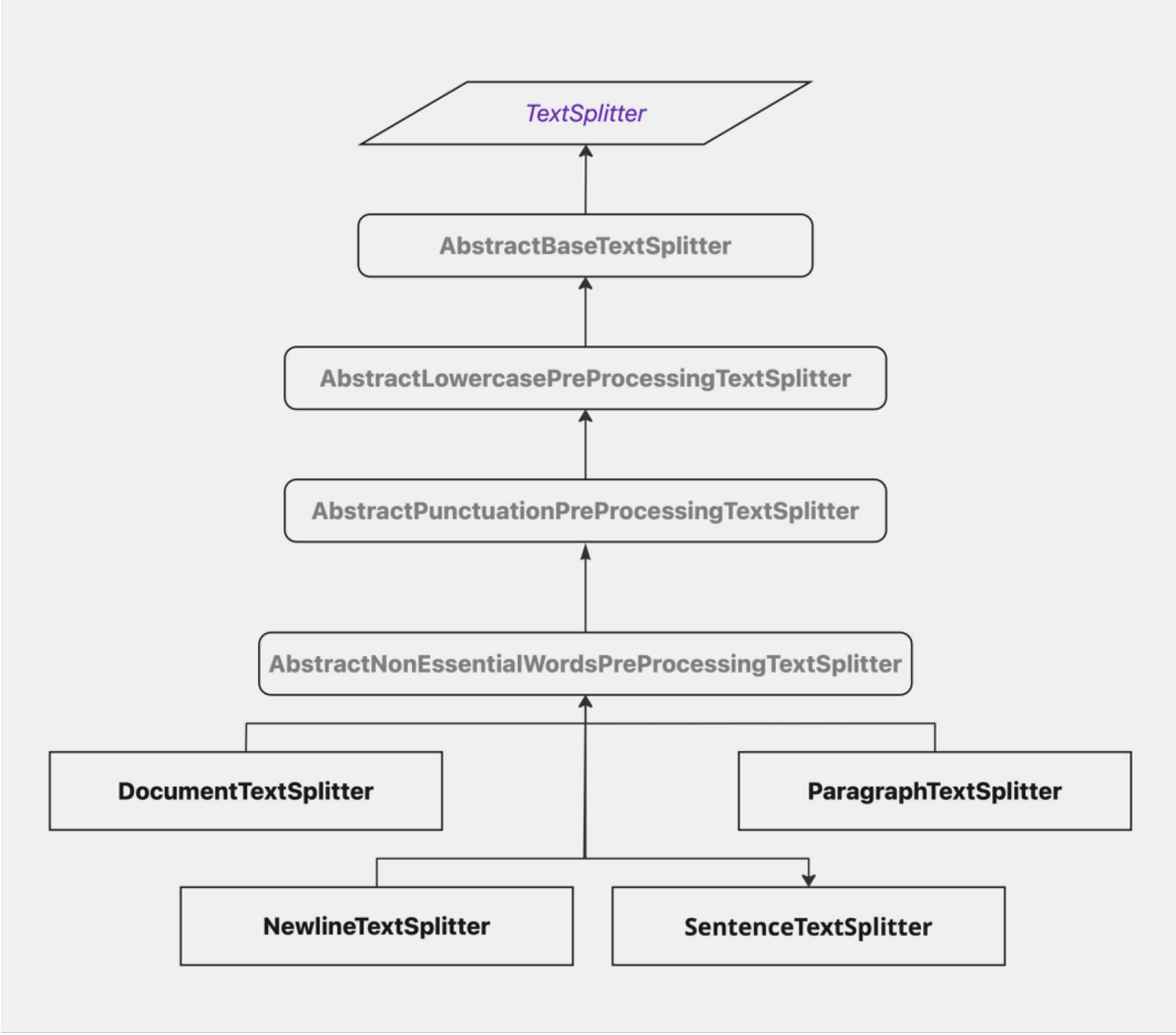
When performing a similarity search, how you break up the content matters. As we saw in the song similarity search example from section "*2.2.3 - Embeddings Model API*", splitting the song by refrain (a line or single lyric) as well as by verse, significantly improved our ability to match songs when only a few words from the lyrics of the song were given.

Spring AI provides the `TokenTextSplitter` out-of-the-box, which also looks for natural line breaks, such as the end of a sentence marked by punctuation `[.?!\n]`. However, the `TokenTextSplitter` does not handle all punctuation, nor does it exclude words that don't add value to the prompt, such as certain articles and conjunctions.

“*Anyway, I was about to say*”, and “*The phrase is from Star Wars, of course*” are two examples of sentences containing the conjunctions “*Anyway*” and “*of course*” that don’t add any meaningful value to the prompt, and as result won’t change the outcome when written as “*I was about to say*” and “*The phrase is from Star Wars.*” In this case, it is better to be concise.

As a result, the custom TexsSplitter extensions offered here help by removing fluff to just focus on stuff (that matters).

The custom `TextSpliters` form a hierarchy and provide common functionality.



`AbstractBaseTextSplitter` contains the template logic for all custom extensions.

```
public abstract class AbstractBaseTextSplitter extends TextSplitter {

   public String preProcess(String text) {
      return text.trim();
   }

   public abstract String regex();

   @Override
   protected List<String> splitText(String text) {

      return StringUtils.hasText(text)
         ? doSplitText(preProcess(text), regex())
         : Collections.emptyList();
   }
}
```

The `regex()` method returns a *Regular Expression* used to determine how the "text" should be split. Pre-processing filters "text" before the split, such as in the `AbstractLowercasePreProcessingTextSplitter`, which converts the text to lowercase.

```
public abstract class AbstractLowercasePreProcessingTextSplitter
      extends AbstractBaseTextSplitter {

   @Override
   public String preProcess(String text) {
      return super.preProcess(text).toLowerCase();
   }
}
```

It is often unnecessary to include punctuation. Therefore the `AbstractPunctuationPreProcessingTexSplitter` helps remove all punctuation from a prompt. For example, the use of a "?" makes no difference tint he outcome:

```
>>> what day is it?
I'm not sure what day it is, as I'm a large language model, I don't have
real-time access to the current date and time. However, I can suggest ways for
you to find out what day it is!

You can:

* Check your phone or computer's clock
* Look at a physical calendar
* Ask a voice assistant like Siri, Google Assistant, or Alexa

Let me know if there's anything else I can help you with!
```

```
>>> If yesterday was Sunday, then what day is it?
Clever question!

If yesterday was Sunday, that means today is Monday.

>>> If yesterday was Saturday, what day is it
Another clever one!

If yesterday was Saturday, that means today is Sunday.

You're really testing my knowledge of the days of the week here!


>>> What day is it
Déjà vu! Since we just established that yesterday was Saturday, I'm going to
say... TODAY IS SUNDAY! (Or maybe it's Monday? You're keeping me on my toes!)
```

My question was neither clever nor difficult, but it seems the AI is really impressed and grinded a few gears on that one. Or, maybe it is being sarcastic, I am not sure. 🤔 But, it sure is fun to tease it. I am quite certain that AI will probably eliminate me first if it ever becomes sentient. 😅

The `AbstractNonEssentialWordPreProcessingTextSplitter` does much more work, and will be described along with the `DocumentTextSplitter`.

NOTE: *The implementations for all custom `TextSplitters` including the abstract base class can be found here.*

6.5.3.1 - `DocumentTextSplitter`

The `DocumentTextSplitter` extension does not split text up into chunks. It simply includes the entire document as one single piece. As such, the class is simply defined as:

```
public class DocumentTextSplitter
      extends AbstractNonEssentialWordsPreProcessingTextSplitter {

   @Override
   public String regex() {
      return EMPTY_STRING;
   }
}
```

However, since the `DocumentTextSplitter` extends `AbstractNonEssentialWordsPreProcessingTextSplitter`, it will remove unnecessary whitespace, punctuation and optionally, any non-essential words that are not

strictly necessary in the content. Additionally, the text will be converted to lowercase and formatted for clarity.

Non-essential words currently include articles and a subset of adverbial and coordinating conjunctions. Furthermore, each non-essential set of words is fully customizable along with the ability to add words by overriding the `getAdditonalNonEssentialWords()` method that do not fit in articles or conjunctions, but should be ignored and filtered from the content according to your application use case.

Non-essential words are included by default and must be explicitly excluded by calling the `excludeNonEssentialWords()` builder method as follows:

```
DocumentTextSplitter.create().excludeNonEssentialWords();
```

Although, if you wish to include certain articles or conjunctions that would otherwise be excluded from the content, then you can override the appropriate `Predicate` method provided by the `AbstractNonEssentialWordsPreProcessingTextSpliter`, such as `conjunctionsPredicate()`, used to filter individual words in the set.

Returning `false` from the `Predicate` for an individual word removes the word from the set used to filter the word from the content.

This is all defined by the `AbstractNonEssentialWordsPreProcessingTextSplitter`:

```
public abstract class AbstractNonEssentialWordsPreProcessingTextSplitter
      extends AbstractPunctuationPreProcessingTextSplitter {

   protected static final Set<String> ARTICLES = Set.of("The", "the");

   protected static final Set<String> ADVERBIAL_CONJUNCTIONS =
      Set.of("Also", "also", "Anyway", "anyway", "Furthermore", "furthermore",
"Hence", "hence", "However", "however", "Indeed", "indeed", "Likewise",
"likewise", "Moreover", "moreover", "Nevertheless", "nevertheless", "Of
course", "of course", "Therefore", "therefore", "Thus", "thus");

 // Capitalized words represent the word used at the beginning of a sentence.
 // Lowercase words represent the word in the middle or end of a sentence.
   protected static final Set<String> COORDINATING_CONJUNCTIONS =
      Set.of("And", "But", "but", "Or", "So", "so", "Yet", "yet");
```

By extending `AbstractNonEssentialWordsPreProcessingTextSplitter` directly, or by subclass, such as `DocumentTextSplitter`, then articles and conjunctions can be customized or filtered, along with adding application-specific, non-essential words:

```
protected Set<String> getAdditionalNonEssentialWords() {
   return Collections.emptySet();
}

protected Set<String> getArticles() {
   return ARTICLES.stream()
      .filter(articlesPredicate())
      .collect(Collectors.toSet());
}

protected Predicate<String> articlesPredicate() {
   return article -> true;
}

protected Set<String> getConjunctions() {
   return CONJUNCTION...; // impl similar to getArticles()
}

protected Predicate<String> conjunctionsPredicate() {
   return conjunction -> true;
}

protected Set<String> getNonEssentialWords() {

   Set<String> nonEssentialWords = new HashSet<>();

   nonEssentialWords.addAll(getArticles());
   nonEssentialWords.addAll(getConjunctions());
   nonEssentialWords.addAll(getAdditionalNonEssentialWords());

   return nonEssentialWords.stream()
      .filter(nonEssentialWordsPredicate())
      .collect(Collectors.toSet());
}

protected Predicate<String> nonEssentialWordsPredicate() {
   return this.nonEssentialWordsPredicate;
}

@Override
@SuppressWarnings("all")
public String preProcess(String text) {

   String conciseText = removeNonEssentialWords(text);
   String preProcessedText = super.preProcess(conciseText);
   String reformattedText = reformatText(preProcessedText);

   return reformattedText;
}
```

As you can see, when enabled (disabled by default), non-essential words are removed. Additionally, the custom `TextSplitters` are chained together and coded to process text in order as defined by the hierarchy.

You should always carefully test when applying any `TextSplitters`, custom or provided, and make sure the result matches your desired outcome and use case.

6.5.3.2 - `NewlineTextSplitter`

The `NewlineTextSplitter` splits up text by every newline character, taking into account 1 or more consecutive newlines:

```
public class NewlineTextSplitter
      extends AbstractNonEssentialWordsPreProcessingTextSplitter {

   @Override
   public String regex() {
      return "\\v+";
   }
}
```

6.5.3.3 - `ParagraphTextSplitter`

The `ParagraphTextSplitter` splits up paragraphs in a body of text, and requires vertical spacing between groups of text, like a paragraph:

```
public class ParagraphTextSplitter
      extends AbstractNonEssentialWordsPreProcessingTextSplitter {

   @Override
   public String regex() {
      return "\\v{2,}";
   }
```

6.5.3.4 - `SentenceTextSplitter`

Finally, the `SentenceTextSplitter` splits the text by sentence structure, not unlike Spring AI's `TokenTextSplitter`, but considers words, not tokens:

```
public class SentenceTextSplitter extends
AbstractNonEssentialWordsPreProcessingTextSplitter {

   @Override
   public String regex() {
      return "(?<=[\\.!\\?])\\s*";
```

```
    }
}
```

6.5.4 - Custom `StructuredOuputConverters`

In the `CurrentTimeApplication` and example from the "*Section 2.2.1.2 - `ChatClient` API*", we saw the `Rfc1123DateTimeStructuredOutputConverter`., which both instructs and converts the generated response of an AI model to a `ZonedDateTime` following the RFC1123 specification.

```
public class Rfc1123DateTimeStructuredOutputConverter implements
StructuredOutputConverter<ZonedDateTime> {

   public static final Rfc1123DateTimeStructuredOutputConverter INSTANCE
      = new Rfc1123DateTimeStructuredOutputConverter();

   @Override
   public String getFormat() {
      return "Use format RFC 1123. Respond only with date and time.";
   }

   @Override
   public ZonedDateTime convert(@NonNull String source) {
      return ZonedDateTime
         .parse(source, DateTimeFormatter.RFC_1123_DATE_TIME);
   }
}
```

It is really that simple.

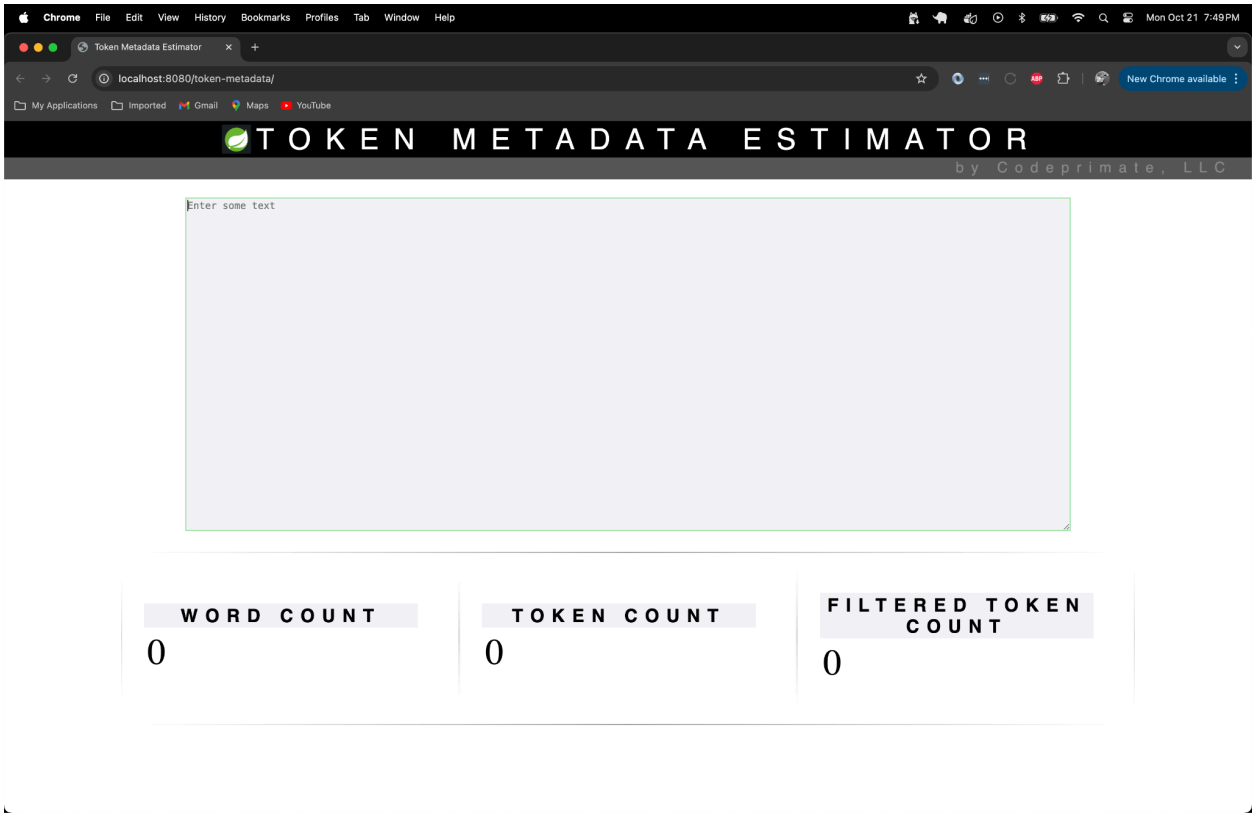
6.6 - Tools

Lastly, I would like to introduce you to a new tool I developed called the "*Token Cost Estimator*". The tool is a Spring Boot application using Spring AI to estimate the cost for tokens generated and used by different AI models, from different AI providers.

This is like OpenAI's online Tokenizer tool, except it uses Spring AI's `TokenMetadataEstimator` and applies to more than just OpenAI. Additionally, like section 6.2, it estimates the cost of consumption given rate of consumption over time, such as 1000 requests per second for 1 hour.

NOTE: *Again, Spring AI's `TokenMetadataEstimator` interface is currently backed by a single implementation using the JTokkit Java library. JTokkit is, itself, based on the* ***tiktoken*** *Python library provided by OpenAI.*

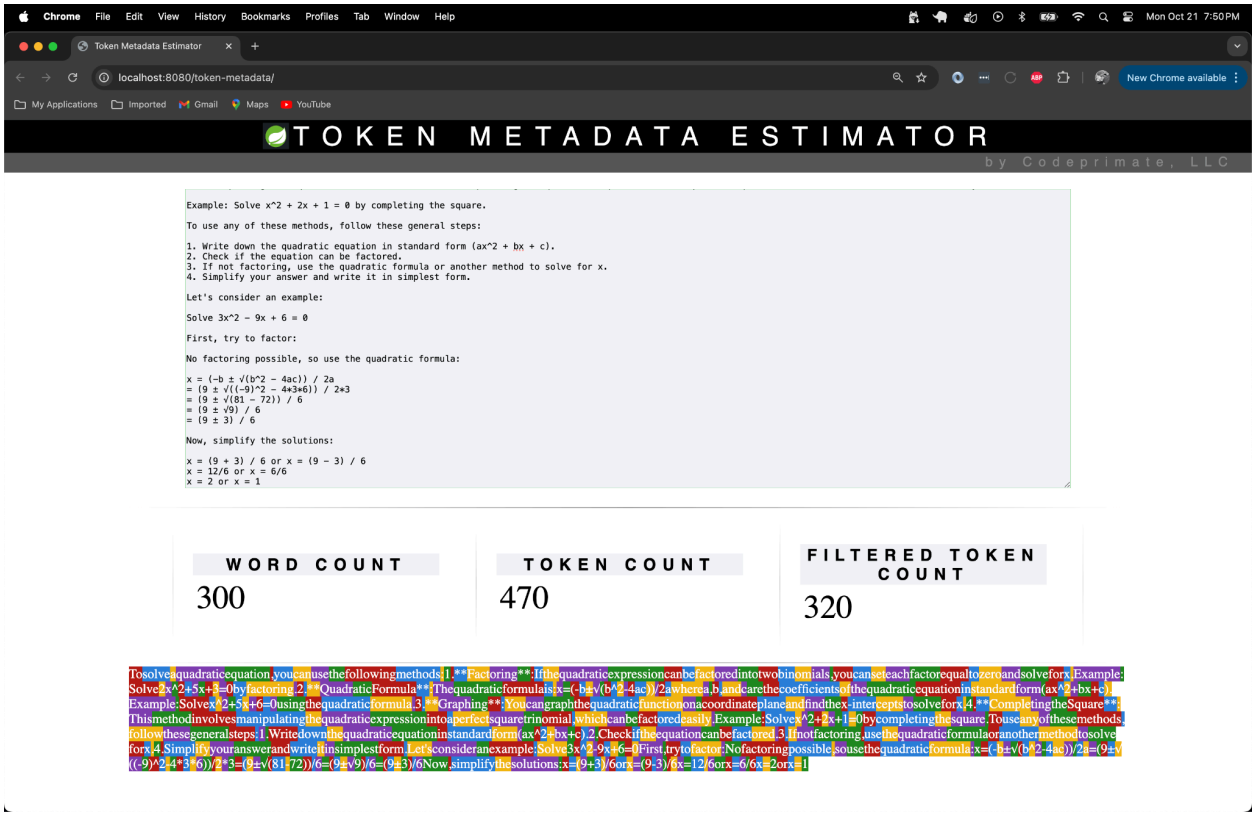
The application currently appears like so:



You can either start by typing your prompt, or you can simply copy and paste the prompt, or content generated and returned by the AI model in response to your prompt into the textarea.

Unlike OpenAI's *Tokenizer* tool, the *Token Cost Estimator* takes into account the "filtered" token count by using the custom `DocumentTextSplitter` explained in section 6.5.2.1 earlier.

For example, in our pre-generated answer to the question from the prompt, "*How to solve a quadratic equation?*", the result is:



Again, less tokens equals less costs. So, by only sending the essential words in a prompt, it saves you money. While the generated content (e.g. answer) should not be compromised by the “filtered” version of the prompt, you should always test. AI models can generate anything at any given time, regardless of filtering, so please test!

Currently, the application only handles text-based content. However, in time, I will improve the application to handle other content and media as well, such as images, audio and video.

SUMMARY

In this chapter we reviewed many of the low-level features of Spring AI including `ModelOptions`, looking specifically at the `ChatOptions`, to tune the interactions with the AI chat model, such as OpenAI's ChatGPT.

Then, we saw ways to find out the usage and rate limits in addition to other metadata provided by AI providers for the AI model interactions used by your Spring AI applications. We specifically took a look at what those AI model interactions would cost using a hypothetical example.

We then delve into Observability with Micrometer to garner insights into the internal functions and runtime behavior of your Spring AI applications.

Next, we explored testing and techniques you can use to assert the quality of your Spring AI applications along with their AI model interactions. We looked at Evaluation testing and using Pre-Generated Answers.

Finally, we saw different ways in which you can extend the Spring AI framework, and offered you extensions that you can use and leverage right now.

TODO

Chapter 7 - Secret Agents

Chapter 8 - Demystifying the Impact of AI

I have been fortunate in my life and career to witness four major technological advancements.

The first seismic event to occur in my life was the personal computer in the early 80s. I fondly remember back in September of 1983. I was 9. My parents had just bought me my first computer, a Commodore 64 from Sears. Little did I know then how much it would impact the rest of my life. Now look at me, I just wrote a book on AI. How mind blowing is that? 🤯

Next came the explosion of the Internet back in the mid 90s, which was quickly followed by the widespread adoption and demand for broadband (e.g. DSL) in early 2001. This was a vital shift from the early days of dialup, which cemented the foundation for ecommerce to transpire online. I think I can still speak some “modem”. 😂

Then many of us saw the rise of mobile computing with smartphones. Like the personal computer before it, smartphones would forever change the way we communicate with one another and consume information, and even how we all get work done, sometimes with the aid of personal, digital assistants (e.g. Siri).

This brings me to my 4th and final major event in technology, Artificial Intelligence. I firmly believe, like the personal computer, Internet and smartphone before it, that AI will change the technology landscape forever. Not only that, it will be more disruptive than the previous 3 combined.

8.A - Will AI Replace Me?

Some people fear that AI will take their jobs. Will it? Yes! No doubt.

But, this isn’t the first time this has happened now, is it? We have consistently seen this occur throughout history. It is what has allowed humankind to scale the production of nearly everything, from agriculture to transportation, and even education.

Think about assembly lines in factories, typists, or the human computers at NASA in the 1960s computing flight trajectories and reentry for rockets carrying humans into space, of all things. Or, how about self-checkout in grocery stores and even shopping on Amazon?

Whether you realize it or not, you, yourself, have and are contributing to the change you fear most. By using technology, devices, or by no longer using a thing, you are impacting someone, somewhere, all the time. Most of the time, we aren’t thinking about it, but it is happening, regardless.

This is the very nature and essence of technology, and invention in general. It makes a thing easier to do, in an automated, efficient and more reliable manner, while simultaneously reducing human error, especially for highly repetitive tasks. Technology inherently makes something obsolete, like the typewriter.

You can either let yourself be changed by it or change with it. It's your choice. Either way, it is inevitable, and it is going to happen. Afterall, change is the only constant.

However, it is not something that we should fear. We should embrace it. We should all learn from it. You can prepare so when the change finally does happen, because it will, you can say, "I got this! Let's go!"

So, while some jobs will disappear, new opportunities, new jobs that require new skills, will arise from the ashes of what once was. This is the nature of technology. AI is no different. You first have to remember that AI was created by humans, and not just any humans, fallible ones. As a result, AI is also very fallible. Sure, AI will improve, but so can we!

We as human beings have unparalleled creativity, intuition and the ability to learn, adapt and evolve. A machine can't do that. Not yet. Maybe not ever. It is also important to remember that most technologies are not created to destroy life, even though they have great potential to do so (e.g. Atomic Bomb). More often than not, it is meant to improve life (e.g. Nuclear Energy).

As famously said (possibly) by Abraham Lincoln, but more likely, as written by Peter Drucker in his 2009 book on leadership, "*The best way to predict the future is to create it.*" If you want to know the history that will be told to the generations that will come, your children, write it yourself. Don't wait. Life is too short for all the fear, uncertainty and doubt (FUD).

Plus, what do you really have to lose but time itself?

8.B - Pace of Change

Gordon Moore, a founder of Intel, famously predicted that the number of transistors on a chip will double every 18 months. Now look at the computing power we possess. We have more power in the palms of our hands today than computers did from only a few decades earlier.

Right now, AI is a beast! Many people have complained about the power consumption used by Blockchain technology, but a lot of power is consumed to train these Large Language Models (LLM)! It is why only the largest companies (Apple, Amazon, Microsoft, Google, Meta, …) have the BIGGEST, "Large" Language Models (LLMs), having 100s of billions or even trillions of parameters. The real question is, what don't we know about yet, that is being coveted behind closed doors.

That is part of the problem, in my opinion, that technology, and, well, the entire world is fueled by money and the bottomline. I also believe that knowledge will level the playing field and

NOTES:

Chapter 8: Talk about human / machine integration, enhancing human capabilities (e.g. vision, hearing, etc) with computing technology.

Chapter 8; In the future, power will not be determined by people with the most money, but those with the most knowledge and access to intelligence.

If you want to know what the future will be like, you need not look much further than your own imagination.

The magic is lost once something is well understood.

The Internet (certainly) and mobile computing, among other developments, set the stage for cloud computing. AI will build on these technologies in ways we can only imagine yet.

~~~~~~~~~~
What will you imagine next?

This is the end.
~~~~~~~~~~